Yükümm

Açık Pazar Yük & Taşımacı Platformu

TEKNİK RAPOR – Ne yapıyoruz ve hangi teknolojilerle?

Backend (.NET 10) • Veritabanı (MSSQL) • iOS (Swift / SwiftUI) • Algoritma ve İş Mantığı

Yapım sırası ve birleştirme: **Yukum_Adim_Adim_Uygulama_Plani.pdf**

FİNAL (Sürüm 3) – araştırmayla güncellendi, tutarlılık taramasından geçti | Rapor tarihi: 20 Ekim 2026 | Kaynak: Apple Notlar – "İŞE GİRİŞ PROJESİ ADINI SEN KOY"

Durum: **Planlama aşaması – kod yazılmadan önceki referans doküman**

Çalışma prensibi: Önce rapor ve algoritma → sonra tasarım seçimi → sonra ekran ekran, adım adım ilerleme.

BAŞLANGIÇ

Raporun Haritası: Ne Nerede?

Bu rapor **Giriş → Gelişme → Sonuç** düzenindedir. Gelişme kısmı alan alan ayrılmıştır: önce **Backend**, sonra **Veritabanı**, sonra **iOS**, sonra **Algoritma ve iş mantığı**. **Yapım sırası ve birleştirme (bağlama)** bu dosyada değil, **Yukum_Adim_Adim_Uygulama_Plani.pdf** dosyasındadır.

Kısımlar ve bölümler

Kısım	Bölüm	Konu	İçinde ne var?
GİRİŞ	1	Yönetici özeti, proje adı	Problem, çözüm, teknoloji özeti
GİRİŞ	2	Notlardaki yönergeler (uyum matrisi)	Notundaki her cümle ve raporda nerede karşılandığı
GİRİŞ	3	Amaç, kapsam, roller	Yük Veren / Taşımacı / Admin, sürüm planı
GİRİŞ	4	Genel sistem mimarisi	iOS ↔ API ↔ DB şeması, ilkeler, ortamlar
GELİŞME I · BACKEND	5	Backend teknik tasarımı	.NET 10 yığını, katmanlı çözüm yapısı, Repository + UoW
GELİŞME I · BACKEND	6	Kimlik doğrulama ve yetkilendirme	İki register, login, JWT, refresh, rol/sahiplik, yetki matrisi
GELİŞME I · BACKEND	7	REST API uç nokta kataloğu	Tüm endpoint'ler, filtre parametreleri, örnek JSON
GELİŞME I · BACKEND	8	Bildirim, e-posta, arka plan işleri	APNs, Outbox, worker, e-posta
GELİŞME II · VERİTABANI	9	Veritabanı tasarımı (MSSQL)	Tablolar, kolonlar, ilişkiler, indeksler, il/ilçe tablosu
GELİŞME III · iOS	10	iOS teknik tasarımı	Swift/SwiftUI yığını, MVVM, klasör yapısı
GELİŞME III · iOS	11	Generic API istemcisi ve auto-login	Tek genel servis metodu, token yenileme, "beni hatırla"
GELİŞME III · iOS	12	Ekran envanteri ve tasarım stratejisi	Ekran listesi, renk/font önerisi, ekran ekran ilerleme
GELİŞME IV · ALGORİTMA	13	Algoritmalar ve durum makineleri	İlan/teklif akışı, kabul, filtre/sıralama, puanlama, sayaç, bildirim
GELİŞME V	14–15	Güvenlik/KVKK/App Store; test, CI/CD, yayın	Kontrol listeleri
GELİŞME VI	16–18	Teknoloji gerekçesi (araştırma), mülakat rehberi, öğrenme planı	Neden bu teknoloji; MVVM/VIPER/TCA; sık sorular
SONUÇ	19	Kaba yol haritası	Faz 0–10 ve Aşamalara eşleşme
SONUÇ	20	Riskler ve açık kararlar	Q listesi, teknik riskler
SONUÇ	21	Değişikliğe dayanıklı tasarım	Neyi değiştirirsek nereye dokunulur, değişiklik protokolü
SONUÇ	22	Sözlük, karar kayıtları	Terimler, K-001…, sıradaki adım

Hangi teknoloji nerede kullanılıyor?

Alan	Kullanılacak teknolojiler	Ayrıntı
Backend	.NET 10 (LTS), ASP.NET Core Web API (controller), EF Core 10 (Code-First, Migrations), Repository + Unit of Work, CQRS-lite handler'lar (MediatR yok), ASP.NET Identity, JWT + rotasyonlu refresh token, FluentValidation, Serilog, OpenTelemetry (temel), yerleşik OpenAPI + Scalar, BackgroundService + Outbox, APNs, SMTP/SendGrid, VS Code	Bölüm 5, 6, 7, 8
Veritabanı	Microsoft SQL Server 2022 (Docker), EF Core migration'ları, `geography` + spatial index, RowVersion (eşzamanlılık), bileşik indeksler, seed (il/ilçe)	Bölüm 9
iOS	Swift 6.2, Xcode 26, SwiftUI, MVVM + Router, Observation (@Observable), async/await + actor, URLSession (generic APIClient), Codable, Keychain, CoreLocation, UserNotifications/APNs, Swift Testing + XCUITest, SPM	Bölüm 10, 11, 12
Altyapı & yayın	Docker Compose, GitHub Actions, Azure (opsiyonel), TestFlight / App Store	Bölüm 14, 15

Sıra, birleştirme, algoritma ve mantık nerede?

Konu	Nerede
Algoritmalar (kabul, filtre, puanlama…) ve **iş mantığı**	Bu rapor, Bölüm 13 (yetki mantığı: Bölüm 6)
Yapım sırası (Aşama A–L, adım adım)	Uygulama Planı, Bölüm 2–3
Birleştirme / bağlama (iOS ↔ API ↔ DB, JSON kuralları, yerel bağlantı)	Uygulama Planı, Bölüm 4–5
Notlar → adımlar eşleşmesi (hiçbir şey atlanmadı kontrolü)	Uygulama Planı, Bölüm 6 ve bu raporun Bölüm 2'si

> **Proje amacı:** Bu proje iş mülakatında "iOS geliştiriciyim ama backend'i ve veritabanını da piyasada rağbet gören güncel teknolojilerle kurdum" diyebilmek için bir vitrin projesidir. Bu yüzden teknoloji seçimleri araştırmayla doğrulandı (Bölüm 16–17).

> **Önemli not:** Notlarda “bunu sakın yapay zekaya *ver yap* mantığıyla vermeyelim” uyarısı var. Bu rapor bu yüzden uygulama kodu değil; **mantık, mimari, algoritma ve yol haritası** içerir. Küçük kod parçaları yalnızca bir kavramı açıklamak için kullanılmıştır. Backend’i yapay zekâ ajanıyla ama kontrollü üreteceksiniz, iOS tarafını ise **siz** öğrenerek yazacaksınız.

GİRİŞ

1. Yönetici Özeti ve Proje Adı

Yüküm, yükü olan kişi/firmaların ilan verdiği, boş dönmek istemeyen taşımacıların ise bu ilanları görüp **teklif verdiği** bir **açık pazar (marketplace)** mobil uygulamasıdır. “Sahibinden” benzeri bir ilan mantığı, “teklif toplama” (açık pazarlık) mantığıyla birleşir.

Çözülen problem (notlardan)

Mersin’de tarım ilaçları taşıtılırken Ankara’dan gelen taşımacı, aynı yolu dönüşte boş yapma riskine karşı fiyatı yükseltiyor (örn. normalde 5 olan fiyata 8–10 diyor). Sebep: **dönüş yükü bulma riski**. Çözüm: yük bulana kadar “pazarda bekleyen” taşımacıların, yakınlarındaki / rotalarındaki yüklere teklif verebildiği şeffaf bir platform. Böylece yük sahibi daha adil fiyat alır, taşımacı boş dönmez.

Teknoloji kararlarının özeti

Katman	Seçim	Gerekçe
Backend	ASP.NET Core Web API – **.NET 10** (LTS), C#	Notta açıkça belirtildi; kurumsal piyasada yaygın; Android/Web de aynı API’yi kullanır.
ORM	**Entity Framework Core 10** + **Repository Pattern** (+ Unit of Work)	Notta zorunlu belirtildi.
Veritabanı	**Microsoft SQL Server (MSSQL)**	Notta belirtildi; `geography` tipi ile konum sorguları.
Kimlik	JWT Access Token + Refresh Token (rotasyonlu), rol/policy tabanlı yetki	Mobil için sektör standardı; ayrı Yük Veren / Taşımacı / Admin rolleri.
Mobil	**Swift 6.2 + SwiftUI** (Xcode 26), MVVM + Router, async/await, URLSession, Keychain	En güncel Apple yığını; iş ilanlarında en çok aranan beceriler.
Bildirim	APNs (push) + e-posta (SMTP/SendGrid)	Notta push ve mail önerildi.
Geliştirme ortamı	Backend: **VS Code + C# Dev Kit** • iOS: **Xcode**	Notta VS Code belirtildi; Xcode iOS için zorunlu.

Proje adı

Notta “adı sen koy / *Yüküm* burada olabilir” denildiği için ana öneri **“Yüküm”**’dür (kısa, Türkçe, “yük” + “benim yüküm” çift anlamı, domain/App Store araması kolay). Alternatifler: *YükPazarı*, *DönüşYük*, *BoşDönme*, *YükBul*. Klasör adı (yükümüTası) da bu yöndedir. Nihai ad **App Store ad müsaitliği ve alan adı** kontrolünden sonra kesinleşmelidir.

GİRİŞ

2. Notlardaki Yönergeler ve Uyarılar (Uyum Matrisi)

Aşağıda notunuzdaki **her** yönerge ayrı ayrı çıkarılmış ve bu raporda nerede karşılandığı gösterilmiştir. Bu tablo proje boyunca kontrol listesi olarak kullanılmalıdır.

2.1 Genel / süreç uyarıları

#	Notlardaki yönerge	Nasıl karşılanıyor	Bölüm
G1	Önce çok iyi bir rapor; algoritma + yol haritası, sonra adım adım git.	Bu doküman; fazlı yol haritası.	13, 19
G2	"Sakın yapay zekaya *ver yap* yaptırma"; mantık, notlar ve rapor oluşturulsun.	Rapor kod değil mantık içerir; her fazın "öğrenme hedefi" ve "siz yapacaksınız" payı tanımlı.	19, 18
G3	Backend'i yapay zekâ/agent ile yap ve öğren; iOS'un mantığı en azından bilinmeli.	Backend için ajan destekli ama gözden geçirmeli üretim süreci; iOS için öğrenerek yazma planı.	5, 19, 18
G4	Tasarımı seç (font, tema), sonra ekran ekran ilerle. Önce çalışır hale getir, sonra süsle.	"Önce işlev, sonra görsel" faz ayrımı; tasarım sistemi kararı Faz 1'de.	12, 19
G5	Konuşarak ilerlet; en sağlam ve güncel teknik detay seçilsin.	Her fazın sonunda karar noktaları; açık sorular listesi.	20
G6	Profesyonel iş akışına benzer, sektöre uygun gerçek dosya/proje yapısı.	Katmanlı backend yapısı ve modüler iOS klasör yapısı.	5, 10
G7	Güncel iş ilanları incelenip en popüler/güncel teknolojiler kullanılsın.	İş ilanı ve kaynak araştırmasıyla doğrulanmış yığın seçimi.	16, 17

2.2 Backend yönergeleri

#	Yönerge	Karşılık	Bölüm
B1	Web API, RESTful servisler; Android + Web + iOS aynı API'yi kullanır.	İstemciden bağımsız (client-agnostic) REST API, JSON, OpenAPI/Swagger.	4, 5, 7
B2	.NET 10 / .NET Core, MSSQL, VS Code ile backend.	Teknoloji tablosu, çözüm yapısı.	1, 5
B3	Login iOS'tan API'ye gider; kontroller API'de; başarılıysa gerekli bilgiler JSON döner.	Login akışı ve response sözleşmesi.	6, 13
B4	Tüm CRUD ve iş işlemleri API tarafında; JSON obje döner; SwiftUI bununla entegre olur.	Tüm iş kuralları servis katmanında; iOS yalnızca DTO tüketir.	5, 7, 10
B5	İki kullanıcı tipi (yük arayan / yük veren), iki ayrı register ekranı/servisi.	İki ayrı register endpoint'i ve ekranı.	6, 7, 12
B6	Tipler birbirinin servis/ekranına girmemeli; API authentication; admin paneline erişim yok.	Rol + policy + sahiplik (ownership) kontrolü.	6
B7	ORM: Entity Framework Core + Repository Pattern.	Generic + özel repository, Unit of Work.	5
B8	İl/ilçe: hazır tablo + servis; il seçilince ilçeler gelsin.	`Cities`, `Districts` tabloları, seed + endpoint.	9, 7

2.3 iOS yönergeleri

#	Yönerge	Karşılık	Bölüm
I1	SwiftUI ve güncel design pattern'ler; en popüler/güncel şirket teknolojileri.	SwiftUI + MVVM + Swift Concurrency + Observation.	10
I2	GET/POST/PUT için genel servis metodu; request/response modeli verilerek her yerden dinamik kullanılsın.	Generic `APIClient` + `Endpoint` soyutlaması.	11
I3	Hazır template veya güzel tasarım; görünüm güzel olsun.	Tasarım sistemi (renk, font, bileşen).	12
I4	Auto-login / "beni hatırla", piyasadaki en sağlam ve güncel yöntem.	Keychain + refresh token rotasyonu + opsiyonel biyometrik.	11
I5	Dosya yapısından proje yapısına gerçek, profesyonel ayrım.	Feature-based modüler klasör yapısı.	10

2.4 İş (fonksiyonel) yönergeleri

#	İstenen özellik	Öncelik	Bölüm
F1	Login / Register (iki tip)	Çekirdek	6, 12
F2	Yük ilanı verme sayfası (yalnızca yük veren)	Çekirdek	13, 12
F3	İlan listeleme sayfası	Çekirdek	12

F4	İlan detay sayfası (ilan sahibi, kendi ilanındaki teklifleri/görüntülenmeyi görür)	Çekirdek	12
F5	İlana teklif verme (yük arayan/taşımacı)	Çekirdek	13
F6	Şehir/ilçe seçimi (il → ilçe cascade)	Çekirdek	9, 7
F7	Filtreleme/sıralama: fiyat, hedef şehir (yük nereye gidiyor), emsal fiyat	Çekirdek	13
F8	Konuma göre filtre: “bana yakın”, “5 km”, “konumuma göre sırala”	Öncelik değil	13
F9	İlan kaydetme / favori	Orta	9, 7
F10	İlan sonrası mail veya push	Orta	8
F11	Görüntülenme ve teklif sayısı tutma (görme öncelikli, teklif sayısı değil)	Öncelik düşük	13
F12	Profil: daha önce anlaşılan işler detaylarıyla; iki tarafın profilinde de geçmiş işler	Orta	9, 12
F13	Push: ilan beğenildiğinde, teklif geldiğinde (ilan sahibine)	Orta	8
F14	Taşımacıya puan/yorum: “yük teslim edildi” statüsünden sonra, yük veren puanlar	Orta	13
F15	Taşımacı profilinde ortalama puan ve geçmiş iş yorumları; puana göre öne çıkarma	Orta	13, 12
F16	Yapay zekâ ile yorum/ilan küfür-yasaklı kelime kontrolü	Opsiyonel / Sonra	8, 14

Notlardaki belirsiz cümleler:
1) **NETLEŞTİ (kullanıcı onayı):** İlanı yük sahibi (iş veren) açar; yük arayan (taşımacı) ilana **fiyat teklifi** verir.
2) “*Diğer kullanıcının kaydettiği ilanlar*” ifadesi: yorum olarak “yük veren, ilanının kaç kişi tarafından favorilendiğini görür ve favorilenince bildirim alır; iki tarafın da profilinde yaptıkları/aldıkları işler görünür.” Doğru değilse düzeltilmeli.

GİRİŞ

3. Proje Amacı, Kapsamı, Ana Fikir ve Roller

3.1 Ana fikir

Yük taşıma sektöründe talep (yük) ile arz (boş araç kapasitesi) çoğunlukla telefon ve tanıdık üzerinden eşleşiyor. Yüküm, bu eşleşmeyi **şeffaf, rekabetçi ve rota odaklı** yapar: yük sahibi ilan açar, taşımacılar teklif verir, yük sahibi en uygun teklifi seçer; iş tamamlanınca taşımacı puanlanır ve itibar (rating) oluşur.

3.2 Neden “açık pazar / açık sermaye”?

- Teklifler (bazı kurallar çerçevesinde) piyasa fiyat algısını şeffaflaştırır; “emsal fiyat” özelliği buradan beslenir.
- Taşımacı yük bulana kadar “pazarda” kalır; platform onların boş dönme riskini azaltır → daha düşük fiyat teklifi.

3.3 Roller

Rol	Notlardaki karşılığı	Yapabilecekleri	Yapamayacakları
Yük Veren (Shipper)	İş veren / yük ilanı veren	İlan oluşturma/düzenleme/kapatma, gelen teklifleri görme, teklif kabul/ret, teslim onayı, taşımacıyı puanlama/yorum, kendi ilanının görüntülenme/favori/teklif sayılarını görme	Teklif verme, başkasının ilanını düzenleme, taşımacı ekranlarına girme
Taşımacı (Carrier)	Yük arayan / iş arayan	İlan arama/filtreleme, favorilere ekleme, teklif verme/geri çekme, anlaşılan işleri görme, teslim durumunu güncelleme, profil ve puan/yorumları görme	İlan açma, yük verenin yönetim ekranlarına girme, kendini puanlama
Admin	Admin kullanıcı	Kullanıcı/ilan/yorum moderasyonu, kullanıcı engelleme, raporları görme	(Normal kullanıcılar admin alanına **asla** giremez)

3.4 Kapsam – Sürüm planı

Sürüm	İçerik
MVP (v1.0)	Register/Login (2 tip), il-ilçe servisi, ilan oluşturma, listeleme, detay, teklif verme/kabul/ret, profil, auto-login, temel filtre/sıralama (fiyat, hedef şehir).
v1.1	Favoriler, push bildirimleri (teklif geldi / beğenildi), e-posta bildirimleri, görüntülenme/teklif sayacı, iş durumu (teslim edildi), puan & yorum.
v1.2	Konuma göre sıralama / “bana yakın X km” (notta öncelik değil), emsal fiyat göstergesi, puana göre öne çıkarma.
v2.0 (sonra)	Yalnızca notta geçen opsiyoneller: yapay zekâ ile yasaklı kelime/içerik kontrolü; Android ve Web istemcilerinin aynı API’ye bağlanması. **Notta olmayan hiçbir özellik eklenmemiştir.**

GİRİŞ

4. Genel Sistem Mimarisi



4.1 Temel ilkeler

- **API-first:** Tüm iş kuralları API'dedir. iOS, Android ve Web sadece sunum yapar (notun "tüm CRUD API'de" kuralı).
- **İstemci güvenilmezdir:** Rol, sahiplik ve doğrulama kontrolleri her zaman sunucuda yapılır; ekranı gizlemek güvenlik sayılmaz.
- **Sözleşme (contract) odaklı:** OpenAPI (Swagger) şeması tek gerçek kaynaktır; iOS modelleri buna birebir uyar. Hata formatı standarttır (RFC 9457 – ProblemDetails).
- **Sürümleme:** `/api/v1/...` — mobil uygulamalar eski sürümlerde kalabildiği için geriye uyumluluk şarttır.

4.2 Ortamlar

Ortam	Amaç	Not
Local	Geliştirme	Mac'te SQL Server için Docker (SQL Server 2022 imajı, Apple Silicon'da emülasyonla). Alternatif: Azure/uzak SQL.
Staging/Test	TestFlight ve entegrasyon testi	Üretimden ayrı veritabanı ve sertifikalar.
Production	Canlı	HTTPS zorunlu, gizli bilgiler Key Vault/ortam değişkeni.

GELİŞME I · BACKEND (.NET)

5. Backend Teknik Tasarımı (.NET 10 Web API)

5.1 Teknoloji yığını

Konu	Seçim	Açıklama
Runtime	.NET 10 (LTS), C# 14	Notta .NET 10 önerildi. Sürüm/SDK kurulumu başlangıçta doğrulanmalı.
Web çerçevesi	ASP.NET Core Web API (Controller tabanlı)	Controller yaklaşımı öğrenme ve sektör uyumu açısından; Minimal API alternatif.
ORM	EF Core 10, Code-First + Migrations	Şema değişiklikleri migration ile versiyonlanır.
Doğrulama	FluentValidation	Request DTO doğrulaması.
Eşleme	Elle mapping veya Mapster/Mapperly	Entity ↔ DTO. AutoMapper ticari lisansa geçtiği için kullanılmaz (Bölüm 16).
Uygulama katmanı	CQRS-lite: Command/Query handler sınıfları (MediatR'sız)	Feature klasörleri; doğrudan DI ile çağrılır.
Kimlik	ASP.NET Core Identity (Guid anahtarlı) + JWT Bearer	Şifre hash'leme, kilitleme, rol yönetimi hazır gelir.
Loglama	Serilog (dosya/console, ileride Seq/App Insights)	Yapısal log, correlation id.
Arka plan işleri	BackgroundService / Hangfire	E-posta, push, ilan süresi dolumu.
Dokümantasyon	Yerleşik OpenAPI (Microsoft.AspNetCore.OpenApi, 3.1) + Scalar arayüzü	iOS tarafı ile sözleşme. Raporlardaki "Swagger" = bu arayüz.
Önbellek	IMemoryCache (il/ilçe, sabit listeler) → ileride Redis	Gereksiz DB yükünü azaltır.
Test	xUnit + Moq/NSubstitute + Testcontainers (SQL Server)	Birim + entegrasyon testi.
Hız sınırlama	ASP.NET Core Rate Limiting	Login ve teklif spam'ine karşı.
Sağlık	Health Checks	DB ve dış servis kontrolü.
Gözlemlenebilirlik	OpenTelemetry (temel izleme)	İstek izi ve metrikler.
Konteyner & CI	Dockerfile + docker-compose (API + SQL Server), GitHub Actions	Tek komutla çalışan ortam; her commit'te build+test.

5.2 Çözüm (Solution) yapısı – Katmanlı / Clean Architecture

```
Yukum.sln
├── src/
│   ├── Yukum.Domain/            # Entity'ler, enum'lar, domain kuralları (bağımlılığı yok)
│   ├── Yukum.Application/       # Servisler, DTO'lar, arayüzler (IRepository, IEmailSender...), validatorlar
│   ├── Yukum.Infrastructure/    # EF Core DbContext, Repository'ler, Migration, Identity, JWT, APNs, SMTP
│   └── Yukum.Api/               # Controller'lar, Middleware, DI, Program.cs, Swagger
└── tests/
    ├── Yukum.UnitTests/
    └── Yukum.IntegrationTests/
```

Application katmanı yapısı: `Features/Loads/Commands/CreateLoad`, `Features/Loads/Queries/GetLoads` … Her kullanım senaryosu bir handler sınıfıdır (CQRS-lite). Komut = durumu değiştirir, sorgu = yalnız okur.

Bağımlılık yönü: Api → Application → Domain; Infrastructure → Application/Domain. Domain hiçbir katmana bağımlı değildir. İş kuralı Application servislerinde, veri erişimi Infrastructure'dadır. Böylece EF Core veya SQL Server ileride değişirse iş mantığı etkilenmez.

5.3 Repository Pattern tasarımı (notta zorunlu)

- **Generic repository:** `IRepository<T>` → Get, List (spec/predicate), Add, Update, Remove.
- **Özel repository'ler:** `ILoadRepository` (konum/filtre sorguları), `IOfferRepository`, `IUserRepository` … Karmaşık sorgular burada kapsüllenir.
- **Unit of Work:** `IUnitOfWork.SaveChangesAsync()` — tek transaction içinde birden çok repository değişikliği (örn. teklif kabul → ilan durumu + diğer teklifleri reddet).
- **Dürüst not:** EF Core'un `DbContext` zaten UoW, `DbSet` repository benzeridir; ek soyutlama bazen fazlalık sayılır. Ancak notta Repository istendiği ve iş görüşmelerinde sık sorulduğu için uygulanacak; **ince (thin) tutulacak**, her şeye sarmalayıcı yazılmayacak.
- Okuma sorgularında `AsNoTracking()`, sayfalamada `Skip/Take` + toplam sayı, N+1 problemine karşı `Include`/projeksiyon.

5.4 İstek yaşam döngüsü

İstek → Rate limit → JWT doğrulama → Role/Policy → Validation → Controller → Service (iş kuralı) → Repository/UoW → DB

- **Global exception middleware:** Beklenmeyen hata → 500 + ProblemDetails (iç detay sızdırmaz); iş kuralı hatası → 4xx.
- **Standart hata gövdesi:** `{ type, title, status, detail, errors{alan:[mesaj]}, traceId }`.
- **Tek tip başarı yanıtı:** Liste uçlarında `{ items, page, pageSize, totalCount }`.

5.5 Konfigürasyon & Gizli bilgiler

- Bağlantı dizesi, JWT imza anahtarı, APNs anahtarı (.p8), SMTP şifresi kod içinde veya Git'te **tutulmaz**; `user-secrets` (local) ve ortam değişkeni/Key Vault (canlı).
- `.gitignore`, `appsettings.{Env}.json` ayrımı.

5.6 Yapay zekâ destekli backend üretim yöntemi (notla uyumlu)

Notta backend'in agent mantığıyla yapay zekâya yaptırılıp **öğrenileceği** yazıyor. Güvenli ve öğretici yöntem:

1. Önce bu rapordaki mimari/ER şeması onaylanır (yapay zekâya "her şeyi yap" denmez).
2. Her seferinde **tek bir dikey dilim** (örn. "Auth", sonra "Cities/Districts", sonra "Loads") istenir.
3. Üretilen kod **okunur**, anlaşılmayan yer sorulur, Swagger'dan elle denenir, test yazdırılır.
4. Her dilim Git'e küçük commit olarak girer.

6. Kimlik Doğrulama ve Yetkilendirme Tasarımı

6.1 Kayıt (iki ayrı akış – notun B5/B6 gereği)

Endpoint	Kim	Alanlar (öneri)	Sonuç
`POST /api/v1/auth/register/shipper`	Yük veren	Ad-soyad / firma adı, e-posta, telefon, şifre, (vergi no – opsiyonel), il/ilçe, KVKK onayı	Kullanıcı `Shipper` rolüyle oluşur
`POST /api/v1/auth/register/carrier`	Taşımacı	Ad-soyad, e-posta, telefon, şifre, araç tipi/kapasitesi, plaka (opsiyonel), çalışma bölgeleri, KVKK onayı	Kullanıcı `Carrier` rolüyle oluşur

Kritik kural: Rol **asla** istemciden "role" alanı olarak gelmez; endpoint'in kendisi rolü belirler. Admin hesabı yalnızca seed/elle veya yalnız admin'in yetkisiyle oluşur — herkese açık kayıt ile admin olunamaz.

6.2 Login akışı (notun senaryosu)

1. iOS → `POST /auth/login` { e-posta/telefon, şifre, deviceId, deviceName }.
2. API: kullanıcıyı bul, şifre hash'ini doğrula, hesap kilidi/aktiflik kontrolü (5 yanlışta geçici kilit).
3. Başarılı ise yanıt: `{ accessToken, accessTokenExpiresAt, refreshToken, user{ id, displayName, role, avatarUrl, ... } }`.
4. iOS tokenları **Keychain**'e yazar, kullanıcıyı rolüne uygun ana ekrana yönlendirir (Yük Veren Home / Taşımacı Home).
5. Başarısız ise: genel mesaj ("E-posta veya şifre hatalı") — hangi alanın yanlış olduğu sızdırılmaz.

6.3 Token stratejisi (sektör standardı)

Token	Ömür	Nerede tutulur	Not
Access Token (JWT)	10–15 dk	iOS: bellek (+Keychain)	Claim'ler: `sub`, `role`, `jti`. Her istekte `Authorization: Bearer`.
Refresh Token	30–90 gün (kayan)	iOS: **Keychain** • DB: **hash'lenmiş**	**Rotasyon:** her yenilemede yeni refresh token verilir, eskisi geçersiz olur. Eski token tekrar kullanılırsa (çalıntı şüphesi) o cihazın tüm token ailesi iptal edilir.

- İmzalama: HS256 (basit) veya RS256/ES256 (ileride çoklu servis). Anahtar gizli yönetilir.
- Çıkış (logout): refresh token sunucuda iptal edilir + Keychain temizlenir. "Tüm cihazlardan çıkış" desteği.
- Şifre değişince tüm refresh token'lar iptal edilir.

6.4 Yetkilendirme matrisi

Kaynak / İşlem	Shipper	Carrier	Admin	Anonim
İl/ilçe listesi	✔	✔	✔	✔ (register ekranı için)
İlan oluştur/düzenle/kapat	✔ (yalnız kendi ilanı)	✘	✔ (moderasyon)	✘
İlan listele/detay	✔	✔	✔	✘ (öneri: giriş zorunlu)
Teklif ver / geri çek	✘	✔	✘	✘
Gelen teklifleri gör, kabul/ret	✔ (yalnız kendi ilanı)	✘	✔	✘
Kendi verdiği teklifler	✘	✔	✔	✘
Favoriye ekle	✔	✔	✘	✘
Taşımacıyı puanla	✔ (yalnız teslim edilen kendi işi)	✘	✘	✘
Admin paneli/uçları	✘	✘	✔	✘

> **İki katmanlı savunma:** (1) `[Authorize(Roles/Policy)]` ile rol kapısı; (2) servis katmanında **sahiplik kontrolü** (örn. `load.ShipperId == currentUserId`). Yalnızca rol kontrolü yetmez — aksi halde bir yük veren başkasının ilanının ID'sini yazarak onu değiştirebilir (IDOR açığı).

GELİŞME I · BACKEND (.NET)

7. REST API Uç Nokta Kataloğu

Tüm yollar `/api/v1` önekiyle başlar. Yanıtlar JSON; liste uçları sayfalıdır (`?page=1&pageSize=20`).

Grup	Metod & Yol	Yetki	Açıklama
Auth	POST /auth/register/shipper	Anonim	Yük veren kaydı
	POST /auth/register/carrier	Anonim	Taşımacı kaydı
	POST /auth/login	Anonim	Giriş, token çifti döner
	POST /auth/refresh	Anonim (refresh token ile)	Token yenile (rotasyon)
	POST /auth/logout	Giriş yapmış	Refresh token iptal
	POST /auth/forgot-password, /reset-password	Anonim	Şifre sıfırlama (e-posta)
	POST /auth/change-password	Giriş yapmış	Şifre değiştir
	DELETE /account	Giriş yapmış	Hesap silme (App Store zorunluluğu)
Referans	GET /cities	Anonim	81 il
	GET /cities/{id}/districts	Anonim	İlin ilçeleri
İlanlar	POST /loads	Shipper	İlan oluştur
	GET /loads	Giriş yapmış	Liste: filtre & sıralama (aşağıda)
	GET /loads/{id}	Giriş yapmış	Detay (+görüntülenme sayar). Sahibine ek: teklifler, sayaçlar
	PUT /loads/{id}	Shipper (sahip)	Güncelle (teklif kabul edilmemişse)
	POST /loads/{id}/close	Shipper (sahip)	İlanı kapat/iptal
	GET /loads/mine	Shipper	Verdiğim ilanlar
	POST /loads/{id}/favorite	Shipper/Carrier	Favoriye ekle
	DELETE /loads/{id}/favorite	Shipper/Carrier	Favoriden çıkar
	GET /loads/favorites	Giriş yapmış	Kaydettiğim ilanlar
Teklifler	POST /loads/{id}/offers	Carrier	Teklif ver
	GET /loads/{id}/offers	Shipper (sahip)	İlana gelen teklifler
	GET /offers/mine	Carrier	Verdiğim teklifler
	POST /offers/{id}/accept	Shipper (sahip)	Kabul → Shipment oluşur
	POST /offers/{id}/reject	Shipper (sahip)	Reddet
	POST /offers/{id}/withdraw	Carrier (sahip)	Geri çek
İşler	GET /shipments/mine	Shipper/Carrier	Anlaşılan işlerim (profil geçmişi)
	GET /shipments/{id}	Taraflar	İş detayı
	POST /shipments/{id}/pickup	Carrier	Yük alındı
	POST /shipments/{id}/deliver	Carrier	Teslim edildi
	POST /shipments/{id}/confirm	Shipper	Teslim onayı → puanlama açılır
Puan/Yorum	POST /shipments/{id}/review	Shipper	Puan + yorum
	GET /carriers/{id}	Giriş yapmış	Taşımacı profili + ortalama puan
	GET /carriers/{id}/reviews	Giriş yapmış	Yorum listesi (sayfalı)
Profil	GET/PUT /me	Giriş yapmış	Kendi profilim
	GET /users/{id}/public-profile	Giriş yapmış	Herkese açık profil
	POST /me/avatar	Giriş yapmış	Profil fotoğrafı
Bildirim	POST /devices	Giriş yapmış	APNs token kaydı
	DELETE /devices/{id}	Giriş yapmış	Token sil (çıkışta)
	GET /notifications	Giriş yapmış	Bildirim kutusu
	POST /notifications/{id}/read	Giriş yapmış	Okundu

Moderasyon (yalnız yayın için)	POST /reports	Giriş yapmış	İlan/yorum/kullanıcı şikâyeti
	POST /users/{id}/block	Giriş yapmış	Kullanıcıyı engelle
	/admin/* (kullanıcı, ilan, yorum, rapor)	**Admin**	Yalnızca admin rolü

7.1 İlan listesi sorgu parametreleri

Parametre	Anlamı
originCityId, originDistrictId	Yük nereden çıkıyor
destCityId, destDistrictId	**Yük nereye gidiyor** (örn. Mersin) – dönüş yükü filtresi
minPrice, maxPrice	Fiyat aralığı (bütçe/emsal)
vehicleType, minWeight, maxWeight, pickupFrom, pickupTo	Uygunluk filtreleri
lat, lon, radiusKm	“Bana yakın” (öncelik değil)
sort	`newest` | `priceAsc` | `priceDesc` | `nearest` | `shipperRating`
q	Başlık/açıklamada arama

7.2 Örnek response sözleşmesi (login)

```
{
  "accessToken": "eyJhbGciOi...",
  "accessTokenExpiresAt": "2026-10-20T10:15:00Z",
  "refreshToken": "base64url-random",
  "user": { "id": "guid", "displayName": "Ali Veli", "role": "Carrier", "avatarUrl": null }
}
```

GELİŞME I · BACKEND (.NET)

8. Bildirim, E-posta ve Arka Plan İşleri

8.1 Bildirim olayları

Notta geçenler: ilan yayınlandı (mail/push), ilana teklif geldi (push), ilan beğenildi/favorilendi (push). Diğer satırlar bu olayların doğal devamıdır; istenirse çıkarılır.

Olay	Kime	Kanal	Sürüm
İlan yayınlandı (onay)	Yük veren	E-posta (+push)	v1.1
İlana yeni teklif geldi	Yük veren	Push + uygulama içi	v1.1
İlan favorilendi (toplu/sınırlı)	Yük veren	Push	v1.1
Teklif kabul / ret edildi	Taşımacı	Push + uygulama içi	v1.1
Teslim edildi / onaylandı	İlgili taraf	Push	v1.1
Yeni yorum aldı	Taşımacı	Push	v1.1

8.2 Teknik notlar

- **APNs:** Token tabanlı kimlik doğrulama (.p8 anahtarı); sunucu HTTP/2 ile APNs'ye bağlanır. Android/Web için ileride FCM — bu yüzden `INotificationSender` soyutlaması kullanılır.
- **İzin:** Push izni, kullanıcıya değer gösterildikten sonra (örn. ilk ilan/teklif sonrası) istenir.
- **Outbox deseni:** Bildirim/e-posta kaydı iş transaction'ı ile birlikte yazılır; ayrı worker gönderir. API'nin yavaşlaması ve "kayıt oldu ama mail gitmedi" sorunu önlenir.
- **E-posta:** SMTP veya SendGrid/Mailgun; şablonlar (HTML) ve Türkçe içerik; SPF/DKIM ayarı canlıda.
- **Zamanlanmış işler:** Süresi dolan ilanları *Expired* yap, süresi dolan teklifleri kapat, eski refresh token'ları temizle.
- **Push içeriği:** Hassas bilgi (fiyat/telefon) push gövdesine yazılmaz; ekranda detay açılır.
- **Derin bağlantı:** Push payload'ında `type` + `entityId`; iOS Router ilgili ekrana yönlendirir.

GELİŞME II · VERİTABANI (MSSQL)

9. Veritabanı Tasarımı (MSSQL)

9.1 Genel kararlar

- **Anahtar:** `Guid` (veya sıralı GUID) – tahmin edilemez ID'ler; liste performansı için `NEWSEQUENTIALID` /UUIDv7 değerlendirilebilir. Referans tablolarda (Cities, Districts) `int`.
- **Para:** `decimal(18,2)` + `Currency` (varsayılan TRY). Asla float.
- **Zaman:** UTC saklanır (`datetime2`), istemci yerel saate çevirir.
- **Ortak alanlar:** `CreatedAt` , `UpdatedAt` , `IsDeleted` (soft delete – ilan/yorum gibi iz bırakması gerekenlerde), `RowVersion` (eşzamanlılık – iki kişi aynı teklifi aynı anda kabul ederse çakışmayı yakalar).
- **Konum:** Enlem/boylam (`float`) + sorgu için `geography` kolonu ve spatial index.
- **Enum'lar:** Veritabanında `tinyint/int` + kod tarafında enum; metin saklamaktan kaçın.

9.2 Varlık-ilişki özeti

```
Users 1──1 ShipperProfiles
Users 1──1 CarrierProfiles ──* Vehicles
Cities 1──* Districts
Users(Shipper) 1──* Loads ──* LoadImages
Loads *──1 (OriginDistrict) / (DestinationDistrict)
Loads 1──* Offers *──1 Users(Carrier)
Loads 1──0..1 Shipments (Offer kabul edilince oluşur) 1──0..1 Reviews
Users *──* Loads  via FavoriteLoads
Loads 1──* LoadViews (tekil görüntülenme)
Users 1──* RefreshTokens
Users 1──* DeviceTokens (push)
Users 1──* Notifications
Users 1──* Reports (şikâyet)
```

9.3 Tablolar ve temel kolonlar

Tablo	Önemli kolonlar	Not
Users (Identity)	Id, Email, PhoneNumber, PasswordHash, FullName, UserType (Shipper/Carrier/Admin), IsActive, IsBlocked, AvatarUrl, EmailConfirmed, CreatedAt	Roller Identity `Roles` / `UserRoles` ile.
ShipperProfiles	UserId (PK/FK), CompanyName, TaxNumber, TaxOffice, About, CityId	Yük veren ek bilgisi.
CarrierProfiles	UserId, About, HomeCityId, ServiceRegions, AverageRating, RatingCount, CompletedJobCount, IsAvailable, CurrentLat/Lon (opsiyonel)	`AverageRating` denormalize saklanır (hız); yorum eklenince güncellenir.
Vehicles	Id, CarrierId, VehicleType, PlateNumber, CapacityKg, CapacityVolume	Araç tipi: tır, kamyon, kamyonet…
Cities	Id, Name, PlateCode, Lat, Lon	81 il, seed.
Districts	Id, CityId, Name, Lat, Lon	~970 ilçe, seed. `GET /cities/{id}/districts`
Loads (İlanlar)	Id, ShipperId, Title, Description, CargoType, WeightKg, Volume, VehicleTypeRequired, OriginCityId, OriginDistrictId, OriginLocation(geography), DestCityId, DestDistrictId, DestLocation, PickupDate, DeliveryDeadline, BudgetPrice (opsiyonel "hedef fiyat"), Currency, Status, ViewCount, OfferCount, FavoriteCount, ExpiresAt, CreatedAt, IsDeleted, RowVersion	Status: Draft/Open/InNegotiation/Assigned/InTransit/Delivered/Completed/Cancelled/Expired.
LoadImages	Id, LoadId, Url, SortOrder	İlan fotoğrafları (v1.1+).
Offers (Teklifler)	Id, LoadId, CarrierId, Price, Currency, Note, EstimatedPickupDate, Status, CreatedAt, UpdatedAt, RowVersion	Status: Pending/Accepted/Rejected/Withdrawn/Expired. **Benzersiz kısıt:** aynı taşımacı aynı ilana tek aktif teklif.
Shipments (Anlaşılan iş)	Id, LoadId, OfferId, ShipperId, CarrierId, AgreedPrice, AgreedAt, Status (Agreed/InTransit/Delivered/Completed/Cancelled), PickedUpAt, DeliveredAt	Profilde "daha önce anlaşılan işler" buradan listelenir.

Reviews	Id, ShipmentId (unique), ReviewerId, CarrierId, Rating (1–5), Comment, IsHidden, CreatedAt	Yalnız *Delivered*/*Completed* işe ve yalnız o işin yük verenine izin.
FavoriteLoads	UserId, LoadId, CreatedAt (PK birleşik)	Kaydedilen ilanlar.
LoadViews	Id, LoadId, ViewerId, ViewedAt (veya günlük tekil)	Tekil görüntülenme; `Loads.ViewCount` sayaç cache'i.
RefreshTokens	Id, UserId, TokenHash, DeviceId, ExpiresAt, RevokedAt, ReplacedByTokenId, CreatedAt	Rotasyon zinciri.
DeviceTokens	Id, UserId, ApnsToken, Platform, DeviceId, LastSeenAt	Push için.
Notifications	Id, UserId, Type, Title, Body, DataJson, IsRead, CreatedAt	Uygulama içi bildirim kutusu.
OutboxMessages	Id, Type, PayloadJson, Status, Attempts, CreatedAt, ProcessedAt	Mail/push'u güvenilir göndermek için.
Reports	Id, ReporterId, TargetType, TargetId, Reason, Status	Yalnız App Store'a yayın yapılırsa gerekli (Apple UGC kuralı); MVP dışı.
AuditLogs	Id, UserId, Action, Entity, EntityId, Before/After, At	Admin ve kritik işlem izi.

9.4 Önemli indeksler

- `Loads(Status, DestCityId, CreatedAt DESC)` — liste + "nereye gidiyor" filtresi.
- `Loads(Status, OriginCityId, CreatedAt DESC)`, `Loads(ShipperId, Status)`.
- Spatial index: `Loads.OriginLocation` — "bana yakın" sorguları.
- Unique: `Offers(LoadId, CarrierId)` (aktif teklif için filtered unique index), `Reviews(ShipmentId)`, `FavoriteLoads(UserId, LoadId)`.
- `Offers(CarrierId, Status)`, `Notifications(UserId, IsRead, CreatedAt DESC)`.

9.5 İl / İlçe tablosu (notun B8 maddesi)

1. `Cities` ve `Districts` tabloları EF Core `HasData` veya SQL seed script ile bir kez doldurulur (resmî il/ilçe listesi kaynak alınır, doğrulanır).
2. API: `GET /cities` ve `GET /cities/{cityId}/districts` — anonim erişilebilir (kayıt ekranında da kullanılır), **cache**'lenir (veri nadiren değişir).
3. iOS: il seçilince ilgili ilçeler API'den çekilir (cascade picker); liste ayrıca cihazda cache'lenebilir.
4. İlan, il ve ilçeyi **Id olarak** saklar (serbest metin değil) → filtreleme/rapor tutarlı olur.

10. iOS Uygulaması Teknik Tasarımı (Swift / SwiftUI)

10.1 Teknoloji yığını (güncel & iş ilanlarında en çok aranan)

Konu	Seçim	Neden
Dil	**Swift 6.2** (Xcode 26, approachable concurrency)	Güncel; veri yarışı güvenliği; App Store Xcode 26 ile derlemeyi şart koşuyor.
UI	**SwiftUI**	Notta istendi; Apple'ın geleceği; ilanlarda baskın.
Min. iOS	**iOS 18** (öneri) – SDK: Xcode 26 / iOS 26	Apple verisi (Haziran 2026): son cihazların ~%86'sı iOS 26, ~%11'i iOS 18. iOS 17 alternatif (Karar Q12). Detay: Bölüm 16.
Mimari	**MVVM** + Router (NavigationStack) + protokol tabanlı DI + servis/repository katmanı	SwiftUI'de varsayılan; ilanlarda en sık geçen. VIPER/TCA anlatılır, uygulanmaz (Bölüm 17).
Eşzamanlılık	**async/await**, `Task`, `actor`, `@MainActor`	Combine yerine modern yaklaşım.
State	`@Observable`, `@State`, `@Bindable`, `@Environment`	Observation framework (modern).
Ağ	**URLSession** (3. parti kütüphane yok)	Sektör tercihi; Alamofire gerekmez.
JSON	`Codable` (+ özel decoder ayarları)	ISO8601 tarih, snake/camel stratejisi.
Güvenli saklama	**Keychain** (token), `UserDefaults` (basit ayar)	Token asla UserDefaults'ta tutulmaz.
Navigasyon	`NavigationStack` + tipli route + Router	Deep link ve push yönlendirmesine uygun.
Konum	CoreLocation (+ MapKit)	"Bana yakın" ve harita.
Bildirim	UserNotifications + APNs	Push.
Yerel veri	SwiftData (opsiyonel; cache/taslak)	MVP'de gerekmez; şehir listesi cache'i için yeterli olabilir.
Görsel	`AsyncImage` / hafif cache	Profil/ilan görselleri.
Test	**Swift Testing** + XCTest (UI)	Birim ve UI testleri.
Bağımlılık	Swift Package Manager	CocoaPods yerine standart.
Kalite	SwiftLint (SPM plugin), SwiftFormat	Kod standardı.
Yerelleştirme	String Catalog (`.xcstrings`)	Türkçe ana dil, ileride EN.

10.2 Katmanlar

View (SwiftUI) → ViewModel (@Observable, @MainActor) → UseCase / Service → Repository (protokol) → APIClient → REST API

- **View:** Sadece görünüm; iş mantığı yok.
- **ViewModel:** Ekran durumu (yükleniyor/başarılı/hata), kullanıcı aksiyonları.
- **Service/Repository:** API çağrısını soyutlar; **protokol** arkasında olduğu için testte sahte (mock) ile değiştirilir.
- **Model:** `DTO` (API şeması) ↔ `Domain model` (ekranın kullandığı). Basit durumlarda tek model kabul edilebilir; büyüdükçe ayrılır.

10.3 Proje / klasör yapısı (feature-based, profesyonel)

```
Yukum/
├── App/
│   ├── YukumApp.swift              # @main, bağımlılık kurulumu
│   ├── AppRouter.swift             # Kök yönlendirme (Auth ↔ Main)
│   └── AppEnvironment.swift        # DI konteyneri / ortam
├── Core/
│   ├── Networking/
│   │   ├── APIClient.swift         # Generic istek motoru
│   │   ├── Endpoint.swift          # path, method, query, body
│   │   ├── APIError.swift
│   │   ├── AuthInterceptor.swift   # Bearer ekleme + 401'de refresh
│   │   └── TokenStore.swift        # Keychain sarmalayıcı
│   ├── Storage/ (KeychainService, UserDefaults+)
│   ├── Location/ (LocationService)
│   ├── Notifications/ (PushManager)
│   └── Utilities/ (Extensions, Formatters, Logger)
├── DesignSystem/
│   ├── Theme/ (Colors, Typography, Spacing)
│   └── Components/ (PrimaryButton, TextFieldStyle, LoadCard, RatingView, EmptyState, ErrorView)
├── Features/
│   ├── Auth/            (Login, RegisterShipper, RegisterCarrier, ForgotPassword)
│   ├── Home/            (Shipper Home, Carrier Home)
│   ├── Loads/           (LoadList, LoadDetail, CreateLoad, Filters)
│   ├── Offers/          (MakeOffer, MyOffers, ReceivedOffers)
│   ├── Favorites/
│   ├── Shipments/       (MyJobs, JobDetail)
│   ├── Reviews/
│   ├── Profile/         (MyProfile, PublicProfile, CarrierProfile)
│   └── Notifications/
│       # Her feature: Views/ ViewModels/ Models/ Services/
├── Resources/ (Assets, Localizable.xcstrings, Info.plist)
└── Tests/ (UnitTests, UITests)
```

Kural: **Feature'lar birbirine doğrudan bağımlı olmaz**, ortak şeyler `Core` / `DesignSystem`'den gelir. Rol ayrımı (Yük Veren / Taşımacı) iki ayrı **TabView** kökü ile yapılır; böylece taşımacı, yük verenin ekranlarına navigasyonla bile ulaşamaz (sunucu kontrolü ise asıl güvenlik).

10.4 Uygulama durumu ve yönlendirme

Durum	Gösterilen
Açılış / token kontrolü	Splash (kısa)
Oturum yok / geçersiz	Login / Register seçimi
Oturum var – rol Shipper	Shipper TabView (Ana sayfa, İlanlarım, Teklifler, Profil)
Oturum var – rol Carrier	Carrier TabView (İlanlar, Favoriler, Tekliflerim/İşlerim, Profil)

10.5 Önemli iOS konuları

- **Hata yönetimi:** Tek `APIError` enum'u (offline, 401, 403, doğrulama, sunucu); kullanıcıya anlaşılır Türkçe mesaj.
- **Yükleme durumları:** `enum LoadState { idle, loading, loaded(T), failed(Error) }`; boş durum ve hata ekranı bileşenleri.
- **Sayfalama:** Sonsuz kaydırma (infinite scroll), pull-to-refresh.
- **Erişilebilirlik:** Dynamic Type, VoiceOver etiketleri, karanlık mod.
- **İzinler:** Konum ve bildirim izni **ihtiyaç anında** istenir; Info.plist açıklamaları Türkçe ve net olur.
- **Gizlilik:** Privacy Manifest (`PrivacyInfo.xcprivacy`) ve App Privacy beyanı.

GELİŞME III · iOS (SWIFT / SWIFTUI)

11. Genel API İstemcisi ve Auto-Login Tasarımı

11.1 Notun isteği

“Ben bu servise gitmek istiyorum, GET/POST dönecek; response ve request modeli bunlar diye **genel bir metod** olmalı, her yerden dinamik kullanılmalı.” Bu, sektörde **generic network layer** olarak bilinir.

11.2 Tasarım

- `Endpoint`**:** Hangi yol, hangi HTTP metodu (GET/POST/PUT/DELETE/PATCH), query parametreleri, ek başlıklar, kimlik gerekli mi (`requiresAuth`). Her feature kendi endpoint’lerini tanımlar (enum veya struct).
- `APIClient` **(protokol + gerçek implementasyon):** Tek bir genel metot — response tipi `Decodable`, request gövdesi `Encodable` — verilerek çağrılır.
- **Interceptor/Adapter:** Bearer token ekler; 401 gelirse **bir kez** token yeniler ve isteği tekrar dener.
- **Token yenileme koordinasyonu:** Aynı anda 5 istek 401 alırsa 5 ayrı refresh atılmaz — `actor` ile tek refresh yapılır, diğerleri sonucu bekler (race condition önlemi).
- **Ortam:** Base URL Debug/Staging/Release için ayrı (xcconfig/Scheme).
- **Gözlemlenebilirlik:** Debug’da istek/yanıt loglanır (token ve şifre maskeli).

11.3 Kavramsal kullanım (açıklama amaçlı, uygulama kodu değildir)

```
// Genel imza (kavram):
func send<Response: Decodable>(_ endpoint: Endpoint, body: Encodable? = nil) async throws -> Response

// Kullanım örnekleri (kavram):
let loads: Page<LoadDTO> = try await api.send(.loads(filter: f))                 // GET
let created: LoadDTO     = try await api.send(.createLoad, body: CreateLoadRequest) // POST
let _: EmptyResponse     = try await api.send(.withdrawOffer(id))                 // POST/PUT, gövdesiz yanıt
```

Böylece yeni bir servis eklemek = yeni bir `Endpoint` tanımı + request/response modeli yazmaktır; ağ kodu tekrar yazılmaz.

11.4 Auto-login / “Beni hatırla” (piyasa standardı)

> Mobilde “beni hatırla” kutucuğu yerine **varsayılan olarak oturumu açık tutmak** standarttır (Instagram, Trendyol vb.). Güvenli yol: kısa ömürlü access token + **Keychain’de** uzun ömürlü **döner (rotating) refresh token**. Şifre cihazda saklanmaz.

Uygulama açılış algoritması

1. Açılışta Keychain’den refresh (ve varsa access) token okunur.
2. Token yoksa → Login ekranı.
3. Access token geçerliyse → doğrudan ana ekran (arka planda `GET /me` ile profil tazelenir).
4. Süresi dolmuşsa → `POST /auth/refresh`. Başarılıysa yeni token çifti Keychain’e yazılır → ana ekran.
5. Refresh reddedilirse (iptal/süre dolmuş/çalıntı şüphesi) → Keychain temizlenir → Login ekranı + “Oturumunuz sona erdi” mesajı.
6. Ağ yoksa → oturum **silinmez**; çevrimdışı durumu gösterilir, bağlantı gelince devam edilir (yanlışlıkla çıkış yaptırmamak için).

Ek güvenlik/UX

- **Keychain erişim sınıfı:** `kSecAttrAccessibleAfterFirstUnlockThisDeviceOnly` (yedekle taşınmaz, arka planda push açılışında okunabilir).
- **Opsiyonel biyometrik kilit:** Face ID / Touch ID (`LocalAuthentication`) ile uygulama açılışında doğrulama — kullanıcı ayarından açılır.
- Uygulama silinip yeniden kurulunca Keychain kalabilir; ilk açılışta bu durum bilinçli ele alınır (isterseniz ilk kurulumda temizlenir).
- Çıkışta: sunucuya logout + push token silme + Keychain temizliği.

12. Ekran Envanteri ve Tasarım Stratejisi

> **Notun kuralı:** “Önce çalışır hale getir, sonra süsle.” — Önce sade/sistem bileşenleriyle çalışan akış, ardından tasarım sistemi uygulanır. Ancak **tasarım kararları (font, renk, tema) baştan seçilir** ki sonradan yeniden yazım gerekmesin.

12.1 Tasarım sistemi önerisi

Öğe	Öneri
Karakter	Güvenilir, sade, “lojistik/yol” hissi; okunabilirlik öncelikli (kullanıcılar sürücü/işletme).
Ana renk	Koyu lacivert (#10243F) – güven; vurgu: turuncu (#F26B1D) – aksiyon/“yol”.
Durum renkleri	Yeşil (kabul/teslim), sarı (bekliyor), kırmızı (ret/iptal).
Yazı tipi	Sistem fontu **SF Pro** (Dynamic Type uyumlu, ek lisans/yük yok) — kimlik için başlıklarda ikinci bir font (örn. SF Pro Rounded) değerlendirilebilir.
Bileşenler	İlan kartı (rota: A → B, fiyat, tarih, araç), durum rozeti, puan yıldızı, boş/hata ekranları, ana buton, filtre sheet.
Mod	Açık + koyu mod; renkler Asset Catalog’da adlandırılmış renk olarak.
İkon	SF Symbols.
Hazır şablon	Notta “hazır template” serbest. Önerilen: yalnızca **görsel ilham** (Figma Community / Dribbble) alıp bileşenleri kendimiz kurmak — hazır kod şablonları öğrenmeyi ve mimariyi bozabilir.

12.2 Ekran envanteri

Alan	Ekran	Rol	Sürüm
Giriş	Splash / oturum kontrolü	Hepsi	MVP
	Karşılama (Yük veriyorum / Taşımacıyım)	Anonim	MVP
	Login	Anonim	MVP
	Register – Yük Veren	Anonim	MVP
	Register – Taşımacı	Anonim	MVP
Yük Veren	Ana sayfa (özet: aktif ilanlar, yeni teklifler)	Shipper	MVP
	İlan Ver (çok adımlı form: yük bilgisi → kalkış/varış il-ilçe → tarih/fiyat → özet)	Shipper	MVP
	İlanlarım + İlan Detay (teklifler, görüntülenme/favori sayacı)	Shipper	MVP
Taşımacı	İlan Listesi (filtre & sıralama)	Carrier	MVP
	İlan Detay + Teklif Ver	Carrier	MVP
	Tekliflerim / İşlerim	Carrier	MVP
	Favoriler	Carrier	v1.1
Ortak	Profilim (bilgi, anlaşılan işler, ayarlar)	Hepsi	MVP
	Taşımacı Profili (ortalama puan + yorumlar)	Hepsi	v1.1
	Bildirimler	Hepsi	v1.1
Filtre/Konum	Filtre sheet (il/ilçe, hedef, fiyat, araç)	Carrier	MVP
	“Bana yakın” (konum izni + yarıçap)	Carrier	v1.2
İş akışı	İş Detayı (durum: alındı / teslim edildi / onay)	İki taraf	v1.1
	Puan ve Yorum Yaz	Shipper	v1.1
Ayarlar	Şifre değiştir, bildirim tercihleri, hesabı sil, çıkış, KVKK/gizlilik	Hepsi	MVP/v1.1

12.3 Ekran ekran ilerleme şablonu (notun yöntemi)

Her ekran için aynı döngü uygulanır:

1. **Ekran kartı:** amaç, rol, kullanılan endpoint’ler, alanlar & doğrulamalar, durumlar (yükleniyor/boş/hata).
2. Backend endpoint’i Swagger’da hazır ve test edilmiş.
3. iOS’ta Endpoint + Model + Service + ViewModel + View (sade).
4. Çalıştır, test et, ardından tasarım sistemi bileşenleriyle güzelleştir.
5. Kabul kriteri sağlandıysa bir sonraki ekran.

13. Çekirdek Algoritmalar ve Durum Makineleri

13.1 İlan durum makinesi (Load)

Draft → Open → InNegotiation → Assigned → InTransit → Delivered → Completed

- **Open:** ilan yayında, teklif alıyor. İlk teklif geldiğinde *InNegotiation* (opsiyonel; sadece göstergedir, yeni teklif almaya devam eder).
- **Assigned:** bir teklif kabul edildi → diğer tüm bekleyen teklifler otomatik *Rejected*; ilan listeden düşer.
- **InTransit:** taşımacı "yükü aldım" der. **Delivered:** taşımacı "teslim ettim" der. **Completed:** yük veren onaylar → puanlama açılır.
- **Cancelled / Expired:** yük veren iptal eder veya `ExpiresAt` geçer (arka plan işi). Atama sonrası iptal kuralı netleştirilmelidir (bkz. Bölüm 20).
- Geçersiz geçişler (örn. Open → Delivered) servis katmanında **reddedilir**.

13.2 Teklif durum makinesi (Offer)

Pending → Accepted | Rejected | Withdrawn | Expired

13.3 İlan oluşturma algoritması

1. İstek JWT'den geldi → rol = Shipper mı? (değilse 403)
2. DTO doğrulama: zorunlu alanlar, ağırlık > 0, tarih geleceğe ait, **il-ilçe uyumu** (ilçe, seçilen ile ait mi?), bütçe ≥ 0.
3. İçerik moderasyonu (v2: AI / basit yasaklı kelime listesi) — şüphe varsa *PendingReview*.
4. Kalkış/varış konumu belirlenir: ilçe merkez koordinatı (veya kullanıcı haritadan işaretlediyse o) → `geography` kolonuna yazılır.
5. Kaydet: Status=Open, ExpiresAt=PickupDate veya +N gün. Sayaçlar 0.
6. Outbox'a "ilan yayınlandı" e-postası/push'u yazılır (aynı transaction).
7. 201 Created + ilan DTO.

13.4 Teklif verme algoritması

1. Rol = Carrier, hesap aktif/engelsiz mi?
2. İlan *Open/InNegotiation* mı ve süresi geçmemiş mi? Taşımacı ilan sahibi değil mi (zaten rol farklı)?
3. Bu taşımacının bu ilanda **aktif teklifi** var mı? Varsa yeni oluşturmak yerine güncelleme (veya ret).
4. Fiyat > 0; makul aralık kontrolü (örn. bütçenin çok üstü/altı için uyarı).
5. Offer kaydı (Pending), `Loads.OfferCount++` (aynı transaction).
6. İlan sahibine push + bildirim kutusu kaydı (Outbox).

13.5 Teklif kabul algoritması (eşzamanlılık kritik)

1. Transaction başlat. İlanı `RowVersion` ile oku.
2. İlan sahibi = istek yapan mı? İlan hâlâ kabul alabilir durumda mı?
3. Seçilen teklif *Accepted*; diğer *Pending* teklifler *Rejected*.
4. İlan → *Assigned*; `Shipment` kaydı oluştur (AgreedPrice = teklif fiyatı).
5. Commit. `DbUpdateConcurrencyException` olursa (iki istek aynı anda) → 409 Conflict.
6. Kazanan taşımacıya ve reddedilenlere bildirim.

13.6 Filtreleme & sıralama algoritması

Sorgu tek bir parametreli LINQ/SQL ile kurulur; filtreler yalnızca verilen parametreler için eklenir (dinamik predicate).

1. Temel koşul: `Status IN (Open, InNegotiation)` ve süresi geçmemiş, silinmemiş, kullanıcının engellemediği yük verenler hariç.
2. **Hedefe göre:** `DestCityId = X` — "Mersin'e giden yükler" (notta: yük nereye gidiyor). İsteğe bağlı olarak kalkış ili ile birlikte: `OriginCityId = Ankara AND DestCityId = Mersin`.
3. **Fiyat:** `BudgetPrice BETWEEN min AND max`; sıralama artan/azalan.
4. **Emsal fiyat:** aynı *(Origin il, Dest il, araç tipi)* için son N gün tamamlanmış işlerin `AgreedPrice` medyanı/ortalaması gösterilir ("bu rotada ortalama ₺X"). Yeterli veri yoksa gösterilmez.
5. **Konum:** `OriginLocation.STDistance(@userPoint) <= radiusKm*1000`; sıralama `nearest` ise STDistance'a göre artan. Hız için önce bounding-box + spatial index. Kullanıcı konumu sunucuda **saklanmaz**, sorgu anında kullanılır.
6. **Puana göre öne çıkarma:** (taşımacıyı listeleyen ekranlarda / yük verenin teklif listesinde) Bölüm 13.7 formülü.
7. Sayfalama: `OrderBy + Skip/Take`, kararlı sıralama için ikinci anahtar olarak `Id`.

13.7 Puanlama algoritması (adil sıralama)

Tek 5 yıldızlı yorumu olan taşımacının, 200 yorumla 4.8 alan taşımacının önüne geçmemesi için **Bayesian (ağırlıklı) ortalama** kullanılır:

```
skor = ( v / (v + m) ) × R  +  ( m / (v + m) ) × C

R = taşımacının ortalama puanı     v = taşımacının yorum sayısı
C = platformun genel ortalaması    m = güven eşiği (örn. 5 yorum)
```

- Yorum yazma koşulu: ilgili *Shipment Delivered/Completed* ve yorum yapan = o işin yük vereni; **iş başına tek yorum** (unique).
- Yorum eklenince `CarrierProfiles.AverageRating/RatingCount` aynı transaction'da güncellenir.
- Taşımacı profili: ortalama puan + yorum listesi + tamamlanan iş sayısı.

13.8 Görüntülenme ve sayaçlar (öncelik düşük)

- Detay açılınca `LoadViews`'a (LoadId, ViewerId, gün) **tekil** kayıt; aynı kullanıcı aynı gün tekrar sayılmaz. İlan sahibi kendi ilanını açarsa sayılmaz.
- `Loads.ViewCount` artırımı atomik (`UPDATE ... SET ViewCount = ViewCount + 1`); yoğunlukta toplu/arka plan sayacı.
- İlan sahibine "X kişi görüntüledi, Y favori, Z teklif" gösterilir. Teklif sayısı önceliklidir değil, görme sayısı önceliklidir (nota göre sıralama: görme > teklif).

13.9 Favori algoritması

1. Ekle: (UserId, LoadId) yoksa ekle (idempotent), `FavoriteCount++`.
2. İlk favoride (ve belirli eşiklerde) ilan sahibine push (spam önlemi: aynı ilan için 1 saatte en fazla N bildirim, toplu: "3 kişi ilanınızı kaydetti").
3. Kaldır: sil, sayaç azalt.

13.10 Bildirim üretim algoritması

Olay (teklif geldi) → Outbox kaydı (aynı transaction) → Arka plan worker → DeviceToken'ları bul → APNs gönder → Başarısızsa tekrar dene (backoff)

Geçersiz (410) APNs token'ları DB'den silinir. Kullanıcı bildirim tercihleri (tür bazında aç/kapa) kontrol edilir.

13.11 Opsiyonel: içerik moderasyonu (öncelik değil)

1. **Aşama 1 (basit):** Yasaklı kelime listesi + normalizasyon (büyük/küçük, Türkçe karakter, boşluk/sembol ile gizleme).
2. **Aşama 2 (AI):** Metin sınıflandırma servisi (ör. bir LLM/moderasyon API'si); sonuç *onayla* / *reddet* / *insan incelemesine al*. Kesinlik yoksa otomatik silme yerine inceleme kuyruğu.
3. Kullanıcı şikâyeti + admin inceleme akışı (zaten App Store için gerekli).

14. Güvenlik, KVKK ve App Store Uyumluluğu

14.1 Güvenlik kontrol listesi

Alan	Önlem
Taşıma	Yalnız HTTPS/TLS; iOS App Transport Security açık (istisna yok).
Şifre	Identity hash (PBKDF2) veya Argon2; minimum uzunluk/karmaşıklık; yanlış denemede geçici kilit.
Token	Kısa ömürlü JWT, rotasyonlu refresh, hash'li saklama, Keychain.
Yetki	Rol + policy + sahiplik kontrolü (IDOR önlemi); admin uçları ayrı policy.
Girdi	Sunucu tarafı doğrulama; EF Core parametreli sorgular (SQL injection önlemi); dosya yükleme türü/boyut sınırı.
Kötüye kullanım	Rate limiting (login, kayıt, teklif); e-posta/telefon doğrulama; bot koruması gerekirse.
Veri	Hassas alanları loglama; telefon/e-posta yalnız anlaşma sonrası karşı tarafa açılabilir (iş modeli kararı).
Altyapı	Gizli bilgiler Key Vault; DB'ye en az yetkili kullanıcı; yedekleme; CORS yalnız bilinen web kaynakları.
Mobil	Jailbreak/TLS pinning değerlendirilebilir (MVP için zorunlu değil); ekran görüntüsü/log'da token yok.
Bağımlılıklar	NuGet/SPM güvenlik güncellemeleri takibi (Dependabot).

14.2 KVKK (Türkiye)

- Aydınlatma metni ve açık rıza (kayıt sırasında onay kutusu + sürüm kaydı).
- Konum verisi: yalnızca amaç için, kullanıcı konumu sunucuda kalıcı tutulmaz; onay alınır.
- Veri silme/dışa aktarma talebi → hesap silme akışı ile uyumlu.
- Telefon/vergi numarası gibi kişisel verilerin sınırlı erişimi ve saklama süresi.

14.3 App Store gereklilikleri (reddi önlemek için)

Konu	Gereksinim
Kullanıcı içeriği (UGC)	İlan/yorum için **şikâyet etme, kullanıcıyı engelleme, filtreleme ve hızlı moderasyon** mekanizması (Guideline 1.2). Bu yüzden Reports/Block uçları MVP'ye yakın tutulmalı.
Hesap silme	Hesap oluşturulabiliyorsa uygulama içinden **silinebilmeli** (Guideline 5.1.1).
Gizlilik	Gizlilik politikası URL'si, App Privacy etiketleri, Privacy Manifest.
İzin metinleri	Konum/bildirim izni için açık ve doğru açıklamalar.
Test hesabı	İnceleme ekibi için hem Shipper hem Carrier demo hesabı.
Giriş yöntemi	Yalnızca kendi e-posta/şifre sistemi → "Sign in with Apple" zorunlu değil. İleride Google/Facebook girişi eklenirse Apple ile giriş de gerekir.
Dağıtım	Apple Developer Programı (yıllık ücretli üyelik) gerekir; test için TestFlight.

Not: Apple ve ilgili mevzuat zamanla değişir; yayın öncesi güncel kurallar yeniden kontrol edilmelidir.

GELİŞME V · GÜVENLİK, TEST, YAYIN

15. Test, Kalite, CI/CD ve Yayın

15.1 Test stratejisi

Katman	Ne test edilir	Araç
Backend birim	Servis iş kuralları: teklif kabul, durum geçişleri, puan hesabı	xUnit
Backend entegrasyon	Gerçek SQL Server ile endpoint testleri, yetki matrisi (403/401 senaryoları)	WebApplicationFactory + Testcontainers
iOS birim	ViewModel, APIClient (sahte URLProtocol), token yenileme yarışı	Swift Testing
iOS UI	Login → ilan ver → teklif ver kritik yolu	XCUITest
Manuel	Swagger / Postman koleksiyonu; gerçek cihaz push testi	—

15.2 CI/CD

- **Git** (GitHub): `main` korumalı; özellik dalları + Pull Request; küçük, anlamlı commit'ler.
- **Backend CI:** build + test + migration kontrolü (GitHub Actions). **CD:** Docker imajı → Azure App Service / bir VPS.
- **iOS CI:** Xcode Cloud veya GitHub Actions (macOS runner): build + test; TestFlight dağıtımı.
- Veritabanı değişiklikleri yalnızca **EF migration** ile; canlıya otomatik uygulama dikkatle (idempotent script).

15.3 Gözlemlenebilirlik

- Serilog + merkezi log; hata izleme (Sentry/App Insights); iOS'ta crash raporu (Xcode Organizer/Sentry).
- Health check endpoint'i ve basit uptime izleme.

16. Güncel Teknoloji Araştırması ve Seçim Gerekçeleri

> **Amaç:** Bu proje iş mülakatında "iOS geliştiriciyim; backend'i ve veritabanını da piyasada rağbet gören güncel teknolojilerle kurdum" diyebilmek için bir vitrin. Bu yüzden notta "en güncel / en çok istenen" denilen her yerde araştırma yapıldı ve seçimler kanıtlandı. Kural: **notta olmayan özellik eklenmedi**; araştırma yalnızca **teknoloji seçimlerini** güncelledi.

Yöntem ve sınırlar: Çok sayıda 2026 iş ilanı ve teknik makale tarandı. Örneklem küçüktür ve kaynakların bir kısmı blog/forumdur (ikincil kaynak); pazar geneli istatistik değildir. Fiyat/lisans gibi hassas bilgiler kullanmadan önce resmî kaynaktan doğrulanmalıdır. Resmî kaynak: Microsoft Learn, Apple Developer.

16.1 iOS tarafı

Konu	Araştırma bulgusu	Karar
UI çerçevesi	SwiftUI, 2026 ilanlarının neredeyse tamamında temel beklenti (Türkiye ilanında: Swift + SwiftUI + REST API + Git).	**SwiftUI** (UIKit yalnız gerekirse).
Mimari	MVVM varsayılan mimari; kaynaklar "MVVM + Coordinator/Router + DI" kombinasyonunu 2026'da varsayılan olarak anlatıyor. VIPER niş (çok büyük ekip/30+ ekran için) ve çoğu uygulama için aşırı karmaşık bulunuyor. TCA uzman/duruma bağlı seçim; ilanlarda nadir. Görüşmeciler desen adından çok **neden seçtiğinizi** açıklayabilmenize bakıyor.	**MVVM + Router (NavigationStack) + protokol tabanlı DI**. VIPER ve TCA mülakatta karşılaştırmalı **anlatılır** (Bölüm 17), projede uygulanmaz. (Karar Q13)
Dil / eşzamanlılık	Swift 6.2: "Approachable Concurrency" ve varsayılan MainActor izolasyonu; Xcode 26'da yeni projelerde varsayılan açık. Async fonksiyonlar çağıran actor'de kalır; arka plan işi için `@concurrent`. Kıdemli ilanlar actor, TaskGroup, protocol-oriented programming istiyor.	**Swift 6.2**, async/await, `actor` (token yenileme), `@MainActor` ViewModel'ler. Combine ana yol değil.
Araç / SDK zorunluluğu	Apple: 28 Nisan 2026'dan beri App Store'a yüklenen uygulamalar **Xcode 26 / iOS 26 SDK** ile derlenmeli. Bu yalnız SDK'yı kısıtlar; desteklenen minimum sürüm ayrı seçilir.	**Xcode 26** ile derleme.
Minimum iOS	Apple verisi (Haziran 2026): son 4 yılda çıkan iPhone'ların ~%86'sında iOS 26, ~%11'inde iOS 18, ~%3'ünde daha eski. `@Observable` ve SwiftData iOS 17 ile geldi (bu bilgi araştırma sonucu değil, mevcut bilgidir; Apple dokümanından doğrulayın).	**Minimum iOS 18** (güncel API'ler, az geriye uyumluluk kodu). Alternatif: iOS 17. (Karar Q12)
State yönetimi	Modern yaklaşım Observation framework.	`@Observable`, `@State`, `@Bindable`, `@Environment`.
Test	XCTest/XCUITest yaygın; Swift Testing yeni ilanlarda geçmeye başlıyor; Instruments nice-to-have.	Swift Testing (birim) + XCUITest (kritik akış) + Instruments (liste performansı).
Ağ katmanı	İlanlar "RESTful API entegrasyonu" istiyor; 3. parti zorunluluk yok.	URLSession + generic APIClient (notun isteği).

16.2 .NET backend ve veritabanı tarafı

Konu	Araştırma bulgusu	Karar
Çalışma zamanı	.NET 10 Kasım 2025'te LTS olarak çıktı; EF Core 10 da LTS, destek Kasım 2028'e kadar, .NET 10 gerektirir (Microsoft Learn).	**.NET 10 + EF Core 10** (notta istenen).
Mimari	Backend ilanlarının çoğunda **Clean Architecture** ve **CQRS** geçiyor; domain odaklı terimler (aggregate, value object, domain event) kıdemli ilanlarda.	Clean Architecture (katmanlı) + Application katmanında **CQRS-lite**: Command/Query handler sınıfları, feature klasörleri. Domain'de temel domain kavramları (enum, value object gerektiği yerde).
Mediator / Mapper kütüphaneleri	MediatR, AutoMapper (ve MassTransit) 2025'te ticari lisansa geçti. MediatR 12.5'e kadar Apache 2.0; 13+ çift lisans (RPL / ticari). İkincil kaynaklara göre küçük gelirli şirket/bireyler için ücretsiz katman var (**resmî lisans metninden doğrulayın**). AutoMapper'da bir 2026 kaynağı yüksek şiddetli açık bildiriyor. Önerilen alternatifler: elle mapping, Mapster/Mapperly.	**MediatR ve AutoMapper kullanılmaz.** Handler'lar doğrudan DI ile çağrılır (gerekirse küçük kendi dispatcher'ımız); mapping elle veya Mapster/Mapperly. Mülakatta "lisans değişikliğini biliyordum ve neden kaçındım" güçlü bir cevap.
API dokümantasyonu	.NET 9'dan itibaren ASP.NET Core şablonları Swashbuckle yerine yerleşik `Microsoft.AspNetCore.OpenApi` paketini kullanıyor; .NET 10'da varsayılan **OpenAPI 3.1**. Etkileşimli arayüz için **Scalar** (`Scalar.AspNetCore`) yaygın seçim.	Yerleşik OpenAPI + **Scalar** arayüzü. (Raporlarda "Swagger" denen yer = bu arayüz.)
Kimlik	İlanlar JWT / OAuth 2.0 / OpenID Connect ve rol tabanlı yetkilendirme (RBAC) istiyor.	JWT + rotasyonlu refresh + RBAC/policy (zaten tasarımda). OAuth2/OIDC kavramı mülakat için öğrenilir; projede sosyal giriş yok (notta yok).
Veri erişimi	EF Core fluent configuration, migrations, performans ayarı ve MSSQL isteniyor.	Fluent API konfigürasyon, migration, indeks ve sorgu planı kontrolü (zaten tasarımda).
Konteyner & CI/CD	Docker ve CI/CD (GitHub Actions / Azure DevOps) ilanlarda sık.	**Dockerfile + docker-compose** (API + SQL Server), **GitHub Actions** ile build+test.
Bulut	Azure çoğu ilanda "güçlü artı" (Key Vault, Identity), bazılarında şart.	Canlı ortam için **Azure** (App Service + Azure SQL + Key Vault) tercih edilebilir; zorunlu değil, kararı canlıya geçişte verilir.

Test	xUnit (birim + entegrasyon) standart.	xUnit + WebApplicationFactory + Testcontainers.
Gözlemlenebilirlik	Serilog, OpenTelemetry/Application Insights, health checks isteniyor.	Serilog + **OpenTelemetry** (temel izleme) + Health Checks.
Arka plan işleri	Hangfire/Quartz.NET çoğunlukla "artı"; MassTransit/Kafka "anlayış" düzeyinde ve MassTransit de ticari.	Yerleşik `BackgroundService` + Outbox deseni. Hangfire gerekirse sonra. Mesaj kuyruğu kullanılmaz (ihtiyaç yok).

Araştırılmayıp mevcut bilgiye dayanan kararlar: Controller tabanlı API (Minimal API ile karşılaştırma doğrulanmadı; mülakat için ikisi de bilinmeli), Redis (MVP'de gerekmiyor, yalnız IMemoryCache). Bunlar araştırma sonucu değildir.

16.3 Kaynaklar

- techcareer.net – iOS geliştirici ilanları; wearedevelopers – iOS/macOS ilanı; hirist.tech – senior iOS ilanı
- iOS App Architecture in 2026: MVVM-C, SwiftUI & Swift 6; Modern iOS App Architecture in 2026: MVVM vs Clean vs TCA
- What is Approachable Concurrency in Xcode 26?; InfoQ – Swift 6.2 Approachable Concurrency
- Apple – SDK minimum requirements; 9to5Mac – iOS 26 adoption (Haziran 2026)
- Microsoft Learn – EF Core 10; Built-in OpenAPI in ASP.NET Core 10; Scalar – ASP.NET Core entegrasyonu
- MediatR ve MassTransit ticari lisansa geçiyor; MediatR vs plain service classes in 2026
- .NET developer ilanı (hirist.tech); EPAM senior .NET ilanı

17. Mülakat Anlatım Rehberi (“iOS’çuyum, backend’i de ben kurdum”)

17.1 30 saniyelik proje anlatımı (taslak)

> “Yüküm, yük sahiplerinin ilan açtığı, taşımacıların ilana fiyat teklifi verdiği bir mobil pazar yeri. iOS tarafını Swift 6 ve SwiftUI ile MVVM mimarisinde yazdım. Backend’i .NET 10 ile Clean Architecture’a uygun katmanlı Web API olarak, veritabanını MSSQL üzerinde EF Core ve Repository Pattern ile kurdum. Kimlik doğrulama JWT ve rotasyonlu refresh token ile, iOS’ta token Keychain’de tutuluyor ve uygulama açılışta otomatik oturum açıyor. Teklif kabulünde eşzamanlılığı RowVersion ile yönettim, bildirimleri Outbox deseniyle APNs üzerinden gönderiyorum.”

17.2 iOS mimari karşılaştırması (MVVM / VIPER / TCA)

Desen	Yapı	Ne zaman	Bu projede
MVC	Model–View–Controller (UIKit’in klasik deseni)	Küçük/eski projeler; “Massive View Controller” sorunu	Kullanılmaz
MVVM	View ↔ ViewModel (durum + aksiyon) ↔ Model/Service. SwiftUI’nin veri bağlama modeline doğal uyum.	SwiftUI’de varsayılan; orta ölçek	**Seçildi** (+ Router + DI)
VIPER	**V**iew (pasif ekran) • **I**nteractor (iş mantığı) • **P**resenter (sunum mantığı) • **E**ntity (model) • **R**outer (yönlendirme). Her ekran için 5 bileşen.	Çok büyük ekip, çok ekran, katı rol ayrımı. Kaynaklara göre küçük/orta projelerde aşırı karmaşık.	Uygulanmaz; **anlatılır**. Yük Veren/Taşımacı akışlarında Interactor ≈ bizim Service/UseCase, Router ≈ bizim Router.
TCA	Tek yönlü veri akışı: State, Action, Reducer, Store (3. parti kütüphane)	Karmaşık, durum ağırlıklı, test odaklı uygulamalar	Uygulanmaz; **anlatılır**
Clean (Domain/Data/Presentation)	Katmanlı, bağımlılık içe doğru	Test edilebilirlik, büyüyen proje	MVVM’in altında hafif uygulanır (Service/Repository protokolleri)

Cevap şablonu: “SwiftUI ve Observation ile MVVM doğal olduğu ve bu ölçekte VIPER’ın getirdiği 5 bileşenli yük gereksiz kaldığı için MVVM seçtim; bağımlılıkları protokol arkasına alıp test edilebilir tuttum. Ekip/ekran sayısı büyürse modülerleşir, hatta TCA veya VIPER değerlendirilir.”

17.3 Sık sorulacak sorular ve cevap iskeleti

iOS

- **@Observable ile ObservableObject farkı?** Observation yalnız okunan özelliği izler → daha az yeniden çizim; `@Published` gerekmez.
- **actor neden?** Token yenileme gibi paylaşılan durumun veri yarışı olmadan tek seferde işlenmesi (5 isteğe 1 refresh).
- **Token nerede saklanır, neden?** Keychain; UserDefaults şifresiz ve yedeğe girer.
- **Retain cycle?** Closure’da `[weak self]`; async/await’te Task yaşam döngüsü.
- **Generic network katmanı nasıl çalışır?** Endpoint + Decodable response + Encodable request; interceptor ile auth.

Backend

- **JWT vs oturum (session)?** Mobil/çok istemci için durumsuz (stateless); kısa ömür + refresh.
- **Refresh token rotasyonu neden?** Çalınan token tek kullanımlık olduğu için tekrar kullanımı tespit edilir → token ailesi iptal.
- **Repository Pattern gerekli mi?** DbContext zaten UoW; dürüst cevap: karmaşık sorguları kapsülleme ve test için ince tutuldu.
- **Rol kontrolü yetmez mi?** Hayır; IDOR için sahiplik kontrolü şart.
- **MediatR neden yok?** Lisans değişikliği; handler’larla CQRS-lite yeterli.
- **Outbox nedir?** Mail/push kaydı iş verisiyle aynı transaction’da yazılır; ayrı worker gönderir → “kayıt oldu ama bildirim gitmedi” olmaz.

Veritabanı

- **Eşzamanlı teklif kabulü?** `RowVersion` (iyimser eşzamanlılık) + transaction → 409.
- **Konum sorgusu?** `geography` + spatial index + `STDistance`.
- **N+1 problemi?** Include/projeksiyon, `AsNoTracking`.
- **Neden decimal para?** Kayan noktalı sayı yuvarlama hatası.
- **İndeks seçimi?** Sık filtrelenen (Status, DestCityId, CreatedAt) bileşik indeks; sorgu planıyla doğrulama.

17.4 Mülakatta gösterilecek paket

- GitHub deposu: iki proje (backend, ios), anlaşılır README, mimari şema, karar kayıtları (K-001…).

- Scalar arayüzünde canlı API; Docker ile tek komutla ayağa kalkan ortam; GitHub Actions rozeti ve geçen testler.
- Demo senaryosu: iki cihazda Yük Veren + Taşımacı → ilan → teklif → push → kabul → teslim → puan.
- Bu raporun Bölüm 16 tablosu: “her seçimin gerekçesi”.

18. İş Görüşmesi İçin iOS Öğrenme/Hazırlık Planı

Notta bu projenin amacı “işe girişte” kullanılmak. Bu nedenle proje, iOS pozisyonlarında sık sorulan konuları kapsayacak şekilde kurgulandı. Her faz bir öğrenme hedefiyle eşleşir:

Konu	Projede nerede	Mülakatta anlatabilmen gereken
Swift temelleri: optional, enum + associated value, protocol, extension, generics, closure, değer/referans tipi	Tüm proje; generic APIClient	struct vs class, protocol-oriented programming
Swift Concurrency: async/await, Task, actor, @MainActor, Sendable	APIClient, token yenileme actor’ü	Veri yarışı nedir, actor neden gerekli
SwiftUI: View yaşam döngüsü, @State, @Binding, @Observable, @Environment, NavigationStack, List/LazyVStack	Tüm ekranlar	State yönetimi, view yeniden çizimi
Mimari: MVVM, katmanlar, DI, protokolle soyutlama	Features/*, Core/*	MVVM vs MVC vs TCA; test edilebilirlik
Networking: URLSession, Codable, hata yönetimi, interceptor, token refresh	Core/Networking	REST, HTTP kodları, 401 akışı
Güvenlik: Keychain, ATS, biyometrik	Auto-login	Token nerede saklanır, neden Keychain
Konum & harita: CoreLocation, MapKit, izin akışı	Faz 8	İzin yaşam döngüsü, pil etkisi
Bildirimler: APNs, UNUserNotificationCenter, deep link	Faz 7	Push mimarisi, token yaşam döngüsü
Kalıcılık: UserDefaults, Keychain, SwiftData, dosya	Cache, ayarlar	Hangisi ne için
Test: birim test, mock, UI test	Faz 3 itibarıyla	Test piramidi, bağımlılık enjeksiyonu
Araçlar: Git, SPM, CI/CD, TestFlight, App Store süreci	Faz 0, 9	Yayın hattı
Performans & bellek: ARC, retain cycle, lazy loading, görsel cache	Liste ekranları	[weak self], Instruments
Backend bilgisi (bonus): REST tasarımı, JWT, EF Core	Faz 2–3	Tam yığın (full-stack) anlatımı güç katar

Portföy ipucu: Projeyi özel bir GitHub deposunda düzenli commit’lerle ilerletin; README’ye mimari şema, ekran görüntüleri ve “neden bu kararı verdim” notları ekleyin. Mülakatta bu raporun karar gerekçeleri doğrudan hazır cevaptır.

SONUÇ · YOL HARİTASI

19. Kaba Yol Haritası (Fazlar) – bağlayıcı sıra Uygulama Planı'ndadır

> **Bağlayıcı yapım sırası Uygulama Planı dosyasındadır (Aşama A–L).** Bu bölümdeki Faz 0–10 kaba çerçevedir. Eşleşme: Faz 0–1 → Aşama A • Faz 2 → B, C • Faz 3 → D • Faz 4 → E • Faz 5 → F (**İlk çalışır hâl = A–F**) • Faz 6 → G, H • Faz 7 → I • Faz 8 → J • Faz 9 → K • Faz 10 → L. Tasarım cilası ve ek özellikler ilk çalışır hâlden sonra eklenir; kesin olmayan kararlar yol ilerledikçe ilgili adımda verilir.

Notun isteği: "Önce olayları anlat, yol haritası, teknik detay ve algoritma; **sonra** adım adım git." Aşağıdaki fazlar sıralıdır; her faz bir **çıkış kriteri** ile biter, sonraki faza ancak onunla geçilir.

Faz 0 – Rapor, kararlar, kurulum Şimdi

- Bu raporun gözden geçirilmesi; Bölüm 20'deki açık kararların konuşulup netleştirilmesi.
- Ürün adı, bundle id (`com.<sizin>.yukum`) ve alan adı kontrolü.
- Kurulumlar: .NET 10 SDK, VS Code + C# Dev Kit, Docker + SQL Server, Git/GitHub, Xcode, Apple Developer hesabı planı, Postman/Swagger.
- **Çıkış kriteri:** Onaylı kapsam (MVP listesi), kurulumlar çalışıyor, boş repo hazır.

Faz 1 – Tasarım kararı ve ekran akışları

- Renk, font, bileşen stili seçimi (Bölüm 12). Wireframe/ekran akış çizimleri (Figma veya kağıt).
- Her MVP ekranı için "ekran kartı" (amaç, endpoint, alanlar, durumlar).
- **Çıkış kriteri:** MVP ekranlarının akışı ve tasarım sistemi onaylı.

Faz 2 – Backend iskeleti ve veritabanı

- Solution/katman yapısı, DI, loglama, ProblemDetails, Swagger.
- EF Core DbContext, Identity, ilk migration; **Cities/Districts seed** ve servisleri.
- Generic Repository + UoW; sağlık kontrolü.
- **Çıkış kriteri:** Swagger'dan il/ilçe uçları çalışıyor, DB migration ile kuruluyor.

Faz 3 – Kimlik (Auth) – hem backend hem iOS

- Backend: iki register, login, refresh (rotasyon), logout, rol/policy, rate limit.
- iOS: proje kurulumu (klasör yapısı), **generic APIClient**, TokenStore (Keychain), Login + iki Register ekranı, auto-login akışı, rol bazlı kök yönlendirme.
- **Çıkış kriteri:** Uygulama kapatılıp açılınca oturum korunuyor; yanlış rol başka alana giremiyor (testle kanıtlı).

Faz 4 – İlan çekirdeği

- Backend: Loads CRUD, listeleme + filtre/sıralama (fiyat, hedef şehir), detay, sahiplik kontrolü.
- iOS: İl/ilçe cascade seçici, İlan Ver (çok adımlı), İlan Listesi, İlan Detay, İlanlarım.
- **Çıkış kriteri:** Yük veren ilan açar; taşımacı listeler/filtreler/detayı görür.

Faz 5 – Teklif akışı

- Backend: Offers, kabul/ret/geri çek, Shipment oluşumu, eşzamanlılık güvenliği.
- iOS: Teklif ver, gelen teklifler (yük veren), tekliflerim/işlerim (taşımacı).
- **Çıkış kriteri:** Uçtan uca: ilan → teklif → kabul → anlaşılan iş. MVP tamamlanır → TestFlight'a ilk iç test sürümü.

Faz 6 – İş durumu, puan/yorum, profil (v1.1)

- Teslim akışı (alındı/teslim/onay), puan & yorum, taşımacı profili, geçmiş işler, görüntülenme/teklif sayacı.
- Favoriler, bildirim kutusu.
- **Çıkış kriteri:** Bir iş baştan sona tamamlanıp puanlanabiliyor; profillerde geçmiş görünüyor.

Faz 7 – Bildirimler (push + e-posta) (v1.1)

- APNs kurulumu, DeviceToken, Outbox + worker, e-posta şablonları, push yönlendirme (deep link).
- **Çıkış kriteri:** Teklif geldi / favorilendi / kabul edildi bildirimleri gerçek cihazda geliyor.

Faz 8 – Konum & emsal fiyat (v1.2)

- Konum izni, "bana yakın / X km / konuma göre sırala", emsal fiyat, puana göre öne çıkarma.

Faz 9 – Güvenlik/uyumluluk sertleştirme ve yayın

- (Yalnız App Store'a yayın yapılırsa) şikâyet/engelleme, hesap silme; gizlilik/KVKK metinleri, Privacy Manifest, performans ve güvenlik taraması, kapsamlı test.
- TestFlight dış test → App Store incelemesi.

Faz 10 – Süsleme ve gelecek (v2)

- Tasarım cilası, animasyonlar; notta opsiyonel olan AI içerik kontrolü; Android/Web istemcileri.

Fazların özet zaman çizelgesi (tahmini)

Faz	Tahmini süre*	Çıktı
0–1	1–2 hafta	Kararlar + tasarım sistemi + ekran kartları
2	1–1,5 hafta	Çalışan API iskeleti + DB
3	2 hafta	Auth uçtan uca + auto-login
4	2–3 hafta	İlan ver / listele / detay
5	2 hafta	**MVP** (teklif akışı)
6–7	3–4 hafta	v1.1 (puan, profil, bildirim)
8	2 hafta	v1.2 (konum/rota)
9	2 hafta	Yayına hazır

*iOS'u öğrenerek yazdığınız ve yan proje olarak ilerlediğiniz varsayımıyla kaba tahmindir; hız, günlük ayrılan zamana göre değişir.

SONUÇ · AÇIK KARARLAR

20. Riskler, Belirsizlikler ve Konuşulacak Kararlar

Notta “konuşarak ilerletilmeli” denildiği için aşağıdaki maddeler, ilgili faza geçmeden önce birlikte karara bağlanmalıdır.

20.1 Açık ürün soruları

#	Soru	Öneri / varsayılan	Karar gerekli faz
Q1	Kim ilan açar?	**KAPANDI:** Yük veren açar, taşımacı fiyat teklifi verir.	—
Q2	Teklifler diğer taşımacılara görünür mü (tam açık pazar) yoksa yalnızca ilan sahibine mi?	Başlangıçta yalnız ilan sahibi görsün; “açık pazar” için toplam teklif sayısı + emsal fiyat göster.	1
Q3	Fiyat: yük veren “hedef/bütçe fiyatı” girer mi, yoksa fiyat tamamen tekliflerle mi oluşur?	Bütçe opsiyonel; boş bırakılabilir.	1
Q4	Karşılıklı pazarlık (karşı teklif) var mı, yoksa tek tur teklif mi?	MVP: tek tur + güncelleme; karşı teklif v2.	1
Q5	Telefon numarası ne zaman paylaşılır?	Anlaşma sonrası açılır.	5
Q8	Anlaşma sonrası iptal kuralları ve cezası?	Cezasız ama kayıt altına alınır, puan etkisi düşünülür.	6
Q9	Yük cinsi kısıtları (tehlikeli madde/ilaç vb. yasal gereklilikler)?	Yük tipi alanı + kullanıcı sorumluluğu şartı; yasal danışmanlık.	4
Q11	Dil: Yalnızca Türkçe mi?	Türkçe; String Catalog ile EN’e hazır.	1
Q12	Min iOS sürümü ve hedef cihazlar?	iOS 18 (araştırma: son cihazların ~%97’si iOS 18+); alternatif iOS 17. iPhone öncelikli.	0
Q13	iOS mimarisi: yalnız MVVM mi, yoksa bir modülde VIPER de denensin mi?	Yalnız MVVM + Router; VIPER/TCA mülakat için öğrenilip anlatılır (Bölüm 17).	0

20.2 Teknik riskler

Risk	Etki	Azaltma
Yapay zekâya “toptan yaptırma” → anlaşılmayan, test edilmemiş kod	Öğrenme hedefi kaybı; bakım zorluğu	Dikey dilimler, kod okuma, test zorunluluğu (Bölüm 5.6).
Kapsam kayması (notta olmayan özelliklerin eklenmesi; konum/AI’nın öne çekilmesi)	MVP gecikir	Sürüm planı (3.4); “öncelik değil” maddeler ertelenir.
Mac’te SQL Server kurulumu zorluğu	Geliştirme gecikmesi	Docker imajı veya uzak/Azure SQL; erken denenmeli.
Token yenileme yarış durumları	Rastgele çıkışlar	Actor ile tek refresh; test yazılması.
Eşzamanlı teklif kabulü	Çift atama	RowVersion + transaction (13.5).
APNs/Apple hesabı gecikmesi	Push testi yapılamaz	Developer hesabı erken açılmalı; Faz 7’den önce hazır.
Hukuki / güven (sahte ilan, dolandırıcılık)	İtibar kaybı	Puan/yorum sistemi, kullanım şartları; yayın yapılırsa Apple gereği şikâyet/engelleme.
Teknoloji sürüm farkları (.NET 10, Xcode/iOS sürümü)	Uyumsuzluk	Kurulumda güncel sürümler doğrulanır; bu rapordaki sürümler başlangıç kararıdır.

SONUÇ · DEĞİŞİKLİK YÖNETİMİ

21. Değişikliğe Dayanıklı Tasarım (İleride Sorun Çıkarmamak İçin)

> **Neden bu bölüm?** Proje yol ilerledikçe şekillenecek: bazı kararlar sonra verilecek, tasarım ve özellikler ilk çalışır hâlden sonra eklenecek. Aşağıdaki kurallar, bu değişikliklerin mevcut çalışan kısmı **bozmadan** yapılabilmesi içindir.

21.1 Neyi değiştirirsek nereye dokunuruz?

Değişecek şey	İzole edildiği yer	Bozmamak için kural
İş kuralı (teklif görünürlüğü, bütçe zorunluluğu, karşı teklif, iptal kuralı)	Application katmanı handler'ları + Domain durum makinesi	Kural tek yerde; controller ve iOS kuralı bilmez, sonucu gösterir. Yeni kural = handler değişikliği + test.
Yeni alan ekleme (örn. ilana yeni bir alan)	Entity + migration + DTO	Alan **opsiyonel/nullable** eklenir; iOS'ta `Optional`. Eski uygulama sürümü bozulmaz.
Yeni enum değeri (yeni ilan durumu)	String enum (API) + `unknown` durumu (iOS)	Bilinmeyen değer iOS'u çökertmez, "güncelle" veya varsayılan görünümle gösterilir.
Kırıcı API değişikliği	`/api/v1` → `/api/v2`	Eski sürüm çalışmaya devam eder; iOS yeni sürüme kendi hızında geçer. Alan silme/yeniden adlandırma yalnız yeni sürümde.
Veritabanı şema değişikliği	Yalnız EF Core migration	**Genişlet–Doldur–Daralt**: önce nullable kolon ekle, sonra veriyi doldur, en son zorunlu yap/eskiyi sil. Doğrudan SQL ile elle değişiklik yok.
Kütüphane/servis değişimi (bildirim, e-posta, token deposu, HTTP istemcisi)	Arayüzler: `INotificationSender`, `IEmailSender`, `IRepository`, iOS'ta `APIClient` / `TokenStore` protokolleri	İş kodu somut sınıfı değil arayüzü bilir; değişim tek implementasyondur.
Tasarım değişimi (renk, font, bileşen)	iOS `DesignSystem/` (Theme + Components) ve Asset renkleri	Ekranlarda sabit renk/font yazılmaz, yalnız tema değerleri kullanılır → tasarım tek yerden değişir.
Minimum iOS sürümü değişimi	Tek build ayarı + `#available` noktaları	Sürüme özel kod bir yerde toplanır.
Yeni istemci (Android/Web)	Aynı REST sözleşmesi (OpenAPI)	İş kuralı backend'de olduğu için istemci yalnız sunum yazar.
Yeni özellik ekleme	Yeni feature klasörü (backend `Features/X`, iOS `Features/X`) + yeni migration + yeni endpoint grubu	Mevcut akışa dokunulmaz; yalnız genişletme noktalarından bağlanır.

21.2 Dokunulmaz çekirdek (kolay kolay değiştirilmeyecekler)

Bunlar projenin güvenliğini ve doğruluğunu taşır; değişmeleri bilinçli bir **karar kaydı** gerektirir:

- Rol endpoint'ten atanır, istemciden gelmez; **her** kaynakta sahiplik kontrolü.
- Refresh token rotasyonu ve hash'li saklama; token'lar iOS'ta Keychain'de.
- Durum makineleri (ilan/teklif/iş) ve geçersiz geçişlerin reddi; teklif kabulünde transaction + RowVersion.
- Veri kuralları: para `decimal`, zaman UTC, kimlikler GUID, enum string, hata formatı ProblemDetails.
- Entity'ler asla doğrudan API'den dışarı verilmez (DTO).
- İş kuralı yalnız backend'de; iOS yalnız gösterir.

21.3 Değişiklik protokolü (her değişiklikte aynı sıra)

1. **Karar kaydı** yaz (K-0xx): ne değişiyor, neden, geri dönüş maliyeti.
2. **Etki analizi:** Uygulama Planı'ndaki eşleşme tablosundan (Bölüm 6) ve adımlardan (Bölüm 3) hangi adımların etkilendiğine bak.
3. **Sözleşme kartını** güncelle (endpoint, DTO, hata kodları); gerekirse sürüm artır.
4. Backend: migration (genişlet–doldur–daralt) → handler → controller → testler → Scalar'da elle deneme.
5. iOS: Endpoint/DTO → ViewModel → View; **eski testler geçmeye devam etmeli**.
6. Git'te küçük commit'ler; her biri geri alınabilir.
7. Raporları (yalnız etkilenen bölümleri) güncelle; karar tablosuna işle.

21.4 Raporlar arası bağlayıcılık

- **Teknik Rapor (bu dosya)** = ne & neden: mimari, veritabanı, API, iOS, algoritmalar, teknoloji gerekçeleri.
- **Uygulama Planı (ayrı dosya)** = hangi sırayla & nasıl bağlanır. **Sıralama ve adım numaralarında Uygulama Planı bağlayıcıdır** (Aşama A–L). Bu raporun Bölüm 19'undaki "Faz 0–10" yalnız kaba çerçevedir ve oradaki tabloyla Aşamalara eşlenmiştir.
- Çelişki olursa: kapsam için **kullanıcının notu**, sıra için **Uygulama Planı**, teknoloji gerekçesi için **bu raporun Bölüm 16'sı** geçerlidir.

SONUÇ · SÖZLÜK VE KARARLAR

22. Ek: Sözlük ve Karar Kayıt Şablonu

22.1 Sözlük

Terim	Anlamı
Yük Veren / Shipper	Taşıtılacak yükü olan ve ilan açan kullanıcı (notlarda iş veren).
Taşımacı / Carrier	Yük arayan, teklif veren araç sahibi (notlarda iş arayan).
Dönüş yükü	Taşımacının boş dönmemesi için bulduğu, kendi güzergâhındaki yük.
Teklif / Offer	Taşımacının bir ilan için verdiği fiyat önerisi.
Shipment	Teklif kabul edilince oluşan, anlaşılmış iş kaydı.
JWT	İmzalı, kısa ömürlü kimlik jetonu.
Refresh token (rotasyonlu)	Yeni access token almak için kullanılan uzun ömürlü, her kullanımda yenilenen jeton.
Repository / Unit of Work	Veri erişimini soyutlayan desen / birden çok işlemi tek transaction'da toplayan desen.
DTO	API ile istemci arasında taşınan veri modeli.
IDOR	Başkasının kaynağına kimliği tahmin ederek erişme güvenlik açığı.
Outbox	Mail/push'u iş verisiyle birlikte güvenle kaydedip sonradan gönderme deseni.
APNs	Apple Push Notification service.
MVVM	Model–View–ViewModel mimarisi.
UGC	Kullanıcı tarafından üretilen içerik (ilan, yorum).

22.2 Karar kayıt şablonu (her önemli kararda doldurulur)

Alan	İçerik
Karar no / tarih	K-001 / ….
Konu	Örn. "Min iOS sürümü"
Seçenekler	A, B, C
Karar & gerekçe	…
Sonuçları / geri dönüş maliyeti	…

22.3 İlk kararlar (bu rapor itibarıyla önerilen)

No	Karar	Durum
K-001	Ürün adı: Yüküm	Öneri – müsaitlik kontrolü bekliyor
K-002	Backend: .NET 10, EF Core, MSSQL, Repository + UoW, katmanlı mimari	Notlardan – onaylı
K-003	Kimlik: JWT + rotasyonlu refresh, rol/policy + sahiplik kontrolü	Öneri
K-004	iOS: Swift 6.2, Xcode 26, SwiftUI, MVVM + Router, async/await, URLSession, Keychain, min iOS 18	Araştırmayla güncellendi
K-005	İlanı yalnız yük veren açar; fiyat teklifini yalnız taşımacı verir	**Kesinleşti**
K-006	MVP kapsamı: Bölüm 3.4'teki v1.0	Öneri
K-007	Backend: Clean Architecture + CQRS-lite; MediatR/AutoMapper yok; yerleşik OpenAPI + Scalar	Araştırmayla belirlendi
K-008	Docker Compose + GitHub Actions + OpenTelemetry (temel)	Araştırmayla belirlendi
K-009	Kapsam: yalnız notlardaki özellikler; ek özellik yok	Kullanıcı kuralı

> **Sonraki adım:** Bu raporu birlikte gözden geçirip Bölüm 20'deki Q1–Q12 sorularını netleştirdikten sonra **Faz 0** → **Faz 1** (tasarım sistemi + ekran kartları) ile başlıyoruz. Kod yazımı, ancak ilgili faz onaylandığında, adım adım ve her adım anlaşılarak yapılacaktır.